

华中科技大学

课程实验报告

课程名称： 数据结构实验

专业班级 CS2404

学 号 U202490035

姓 名 阮云轩

指导教师 叶晨成

报告日期 2025 年 6 月 12 日

计算机科学与技术学院

目 录

1 基于顺序存储结构的线性表实现.....	1
1.1 实验目的	1
1.2 线性表运算定义	1
1.3 问题描述	3
1.4 系统设计	3
1.5 系统实现	3
1.6 系统测试	15
1.7 实验小结	24
2 基于邻接表的图实现	25
3 实验目的	25
4 基于顺序存储结构的线性表实现.....	26
4.1 问题描述	27
4.2 系统设计	28
4.3 系统实现	28
4.4 系统测试	47
4.5 实验小结	56
参考文献	58
附录 A 基于顺序存储结构线性表实现的源程序	59
附录 B 基于链式存储结构线性表实现的源程序	75
附录 C 基于二叉链表二叉树实现的源程序	95
附录 D 基于邻接表图实现的源程序.....	119

1 基于顺序存储结构的线性表实现

线性表是最常用且最简单的一种数据结构。在非空线性表中，每一非首结点都有其前驱；同时，每一非尾结点都有其后继。在非空表中每一个元素都有一个确定的位置，即如果 $a[i]$ 为第 i 位数据元素，我们称 i 为 $a[i]$ 在线性中的位序。事实上，线性表是一个相当灵活的数据结构，长度可以按需改变，同时元素不仅可以访问，还可以增删等。也就是我们常说的增删改查。

1.1 实验目的

- 1) 加深对线性表的概念、基本运算的理解；
- 2) 熟练掌握线性表的逻辑结构与物理结构的关系；
- 3) 物理结构采用顺序表，熟练掌握顺序表基本运算的实现。

1.2 线性表运算定义

依据最小完备性和常用性相结合的原则，以函数形式定义了线性表的初始化表、销毁表、清空表、判定空表、求表长和获得元素等 12 种基本运算，具体运算功能定义如下：

1. 初始化表：函数名称是 `InitList(L)`；初始条件是线性表 L 不存在；操作结果是构造一个空的线性表；
2. 销毁表：函数名称是 `DestroyList(L)`；初始条件是线性表 L 已存在；操作结果是销毁线性表 L ；
3. 清空表：函数名称是 `ClearList(L)`；初始条件是线性表 L 已存在；操作结果是将 L 重置为空表；
4. 判定空表：函数名称是 `ListEmpty(L)`；初始条件是线性表 L 已存在；操作结果是若 L 为空表则返回 `TRUE`，否则返回 `FALSE`；
5. 求表长：函数名称是 `ListLength(L)`；初始条件是线性表已存在；操作结果是返回 L 中数据元素的个数；
6. 获得元素：函数名称是 `GetElem(L, i, e)`；初始条件是线性表已存在， $1 \leq i \leq \text{ListLength}(L)$ ；操作结果是用 e 返回 L 中第 i 个数据元素的值；
7. 查找元素：函数名称是 `LocateElem(L, e, compare())`；初始条件是线性表已存

在；操作结果是返回 L 中第 1 个与 e 满足关系 compare () 关系的数据元素的位序，若这样的数据元素不存在，则返回值为 0；

8. 函数名称是 PriorElem(L,cure,prece); 初始条件是线性表 L 已存在；操作结果是若 cure 是 L 的数据元素，且不是第一个，则用 prece 返回它的前驱，否则操作失败，prece 无定义；
9. 获得后继：函数名称是 NextElem(L,cure,nexte); 初始条件是线性表 L 已存在；操作结果是若 cure 是 L 的数据元素，且不是最后一个，则用 nexte 返回它的后继，否则操作失败，nexte 无定义；
10. 插入元素：函数名称是 ListInsert(L,i,e); 初始条件是线性表 L 已存在, $1 \leq i \leq \text{ListLength}(L)+1$ ；操作结果是在 L 的第 i 个位置之前插入新的数据元素 e。
11. 删除元素：函数名称是 ListDelete(L,i,e); 初始条件是线性表 L 已存在且非空, $1 \leq i \leq \text{ListLength}(L)$ ；操作结果：删除 L 的第 i 个数据元素，用 e 返回的值；
12. 遍历表：函数名称是 ListTraverse(L,visit()), 初始条件是线性表 L 已存在；操作结果是依次对 L 的每个数据元素调用函数 visit()。

附加功能：

1. 最大连续子数组和：函数名称是 MaxSubArray(L); 初始条件是线性表 L 已存在且非空，请找出一个具有最大和的连续子数组（子数组最少包含一个元素），操作结果是其最大和；
2. 和为 K 的子数组：函数名称是 SubArrayNum(L,k); 初始条件是线性表 L 已存在且非空, 操作结果是该数组中和为 k 的连续子数组的个数；
3. 顺序表排序：函数名称是 sortList(L); 初始条件是线性表 L 已存在；操作结果是将 L 由小到大排序；
4. 实现线性表的文件形式保存：其中，□ 需要设计文件数据记录格式，以高效保存线性表数据逻辑结构 (D,R) 的完整信息；□ 需要设计线性表文件保存和加载操作合理模式。附录 B 提供了文件存取的参考方法。注：□ 保存到文件后，可以由一个空线性表加载文件生成线性表。
5. 实现多个线性表管理：设计相应的数据结构管理多个线性表的查找、添加、移除等功能。

* 注：附加功能（5）实现多个线性表管理应能创建、添加、移除多个线性表，单线性表和多线性表的区别仅仅在于线性表管理的个数不同，多线性表

管理应可自由切换到管理的每个表，并可单独对某线性表进行单线性表的所有操作。

1.3 问题描述

采用顺序表作为线性表的物理结构，实现 1.2 小节的运算。其中 ElemType 为数据元素的类型名，具体含义可自行定义。框架设计上构造一个具有菜单的功能演示系统。其中，在主程序中完成函数调用所需实参值的准备和函数执行结果显示，并给出适当的操作提示显示。附录 A 提供了简易菜单的框架。

1.4 系统设计

首先为在 C 语言下实现上述功能，我们大致可以分为三个部分，主程序框架、多表管理框架菜单、对当前表操作框架菜单

主程序框架负责实现多表管理和对当前表操作的框架切换和退出终止的操作。

而多表管理则是负责多个顺序表的读写操作，那么则需是编号或命名等操作去区分不同的表，因此我们需要设计一个 struct 结构去装下这些信息。

而最后一个对当前表操作则是对传入的顺序表 L 去进行线性表基本运算等操作，负责装下这些函数和选择调用。

1.5 系统实现

在设计这么一个菜单框架时我们可以通过多种的方法去实现，那么我的方法是通过 switch() 函数和 goto 语法实现在多个 switch-while 框架中的切换，除此之外我们也可以通过函数指针去实现此菜单功能，然后利用 system("PAUSE"); 实现了消除了每次操作后的「清屏」，使得菜单功能选项仍然出现在屏上不需改变位置。

随后的则是建立多表操作如建立表，删除表，查找并选择表，返回主菜单判断是要继续对表操作还是终止操作，即对此表操作。

而在当前表操作时则是在加入多个基本运算之外还要加一个返回主菜单操作。

1.5.1 初始化表

1) 函数基本说明

函数名称是 InitList(L); 初始条件是线性表 L 不存在; 操作结果是构造一个空的线性表 (35);

Listing 1 目标程序

```
1      status InitList(SqList& L)
2      {
3          if(L.elem == NULL)
4          {
5              L.elem=(ElemType *) malloc(sizeof(ElemType)*LIST_INIT_SIZE);
6              L.length=0;
7              L.listsize=LIST_INIT_SIZE;
8              return OK;
9          }
10         else
11         {
12             return INFEASIBLE;
13         }
14     }
```

1.5.2 销毁表

1) 函数基本说明

函数名称是 DestroyList(L); 初始条件是线性表 L 已存在; 操作结果是销毁线性表 L(2);

Listing 2 目标程序

```
1      status DestroyList(SqList& L)
2      {
3          if(L.elem!=NULL)
4          {
5              free(L.elem);
6              L.elem = NULL;
7              L.length = 0;
8              L.listsize = 0;
9              return OK;
10         }
```

```
11         else
12         {
13             return INFEASIBLE;
14         }
15     }
```

1.5.3 清空表

1) 函数基本说明

函数名称是 ClearList(L); 初始条件是线性表 L 已存在; 操作结果是将 L 重置为空表 (3);

Listing 3 目标程序

```
1     status ClearList(SqList& L)
2     {
3         if(L.elem!=NULL)
4         {
5
6             L.length=0;
7             return OK;
8         }
9         else
10        {
11            return INFEASIBLE;
12        }
13    }
```

1.5.4 判定空表

1) 函数基本说明

函数名称是 ListEmpty(L); 初始条件是线性表 L 已存在; 操作结果是若 L 为空表则返回 TRUE, 否则返回 FALSE(4);

Listing 4 目标程序

```
1     status ListEmpty(SqList L)
2     {
3         if(L.elem==NULL)
4         {
```

```
5             return INFEASIBLE;
6         }
7         if(L.length==0)
8         {
9             return TRUE;
10        }
11        else
12        {
13            return FALSE;
14        }
15    }
```

1.5.5 求表长

1) 函数基本说明

函数名称是 ListLength(L); 初始条件是线性表已存在; 操作结果是返回 L 中数据元素的个数 (5);

Listing 5 目标程序

```
1     status ListLength(SqList L)
2     {
3         if(L.elem!=NULL)
4         {
5             return L.length;
6         }
7         else
8         {
9             return INFEASIBLE;
10        }
11    }
```

1.5.6 获得元素

1) 函数基本说明

函数名称是 GetElem(L,i,e); 初始条件是线性表已存在, $1 \leq i \leq \text{ListLength}(L)$; 操作结果是用 e 返回 L 中第 i 个数据元素的值 (6);

Listing 6 目标程序


```
1      status GetElem(SqList L,int i,ElemType &e)
2      {
3          if(i<1||i>L.length)
4          {
5              return ERROR;
6          }
7          if(L.elem!=NULL)
8          {
9              e=L.elem[i-1];
10             return OK;
11         }
12         else
13         {
14             return INFEASIBLE;
15         }
16     }
```

1.5.7 查找元素

1) 函数基本说明

函数名称是 LocateElem(L,e,compare()); 初始条件是线性表已存在; 操作结果是返回 L 中第 1 个与 e 满足关系 compare() 关系的数据元素的位序, 若这样的数据元素不存在, 则返回值为 0(7);

Listing 7 目标程序

```
1      int LocateElem(SqList L,ElemType e)
2      {
3          if(L.elem!=NULL)
4          {
5              for(int i=0;i<L.length;i++)
6              {
7                  if(e==L.elem[i])
8                  {
9                      return i+1;
10                 }
11             }
12             return 0;
13
14             return OK;
```

```
15         }
16     else
17     {
18         return INFEASIBLE;
19     }
20 }
```

1.5.8 获得前驱

1) 函数基本说明

函数名称是 PriorElem(L,cure,pree); 初始条件是线性表 L 已存在; 操作结果是若 cure 是 L 的数据元素, 且不是第一个, 则用 pree 返回它的前驱, 否则操作失败, pree 无定义 (8);

Listing 8 目标程序

```
1      status PriorElem(SqList L,ElemType e,ElemType &pree)
2      {
3          if(L.elem!=NULL)
4          {
5              for(int i=1;i<L.length;i++)
6              {
7                  if(e==L.elem[i])
8                  {
9                      pree=L.elem[i-1];
10                     return OK;
11                 }
12             }
13             return ERROR;
14         }
15     else
16     {
17         return INFEASIBLE;
18     }
19 }
20 }
```

1.5.9 获得后继

1) 函数基本说明

函数名称是 NextElem(L,cure,nexte); 初始条件是线性表 L 已存在; 操作结果是若 cure 是 L 的数据元素, 且不是最后一个, 则用 nexte 返回它的后继, 否则操作失败, nexte 无定义 (9);

Listing 9 目标程序

```
1      status NextElem(SqList L,ElemType e,ElemType &next)
2      {
3          if(L.elem!=NULL)
4          {
5              for(int i=1;i<L.length-1;i++)
6              {
7                  if(e==L.elem[i])
8                  {
9                      next=L.elem[i+1];
10                     return OK;
11                 }
12             }
13             return ERROR;
14         }
15         else
16         {
17             return INFEASIBLE;
18         }
19     }
20 }
```

1.5.10 插入元素

1) 函数基本说明

函数名称是 ListInsert(L,i,e); 初始条件是线性表 L 已存在, $1 \leq i \leq \text{ListLength}(L)+1$; 操作结果是在 L 的第 i 个位置之前插入新的数据元素 e(10)。

Listing 10 目标程序

```
1      status ListInsert(SqList &L,int i,ElemType e)
2      {
```

```
3         if(L.elem==NULL) return INFEASIBLE;
4         if(i<1 || i>L.length+1) return ERROR;
5         if(L.length>=L.listsize){
6             int *newbase=(int
7                 *)realloc(L.elem,(L.listsize+1)*sizeof(int));
8             if(newbase==NULL) return 0;
9             L.listsize++;
10            L.elem=newbase;
11        }
12        //printf(" ");
13        for(int j=L.length;j>i-1;j--){
14            L.elem[j]=L.elem[j-1];
15        }
16        L.elem[i-1]=e;
17        L.length++;
18        return OK;
19    }
```

1.5.11 删除元素

1) 函数基本说明

函数名称是 ListDelete(L,i,e);初始条件是线性表 L 已存在且非空, $1 \leq i \leq \text{ListLength}(L)$;
操作结果: 删除 L 的第 i 个数据元素, 用 e 返回的值 (11);

Listing 11 目标程序

```
1     status ListDelete(SqList &L,int i,ElemType &e)
2     {
3         if(L.elem==NULL)
4         {
5             return INFEASIBLE;
6         }
7         if(i<=0 || i>L.length)
8         {
9             return ERROR;
10        }
11        e=L.elem[i-1];
12        i--;
13        for(i<=L.length-1;i++)
14        {
15            L.elem[i]=L.elem[i+1];
```

```
16         }
17         L.length--;
18         return OK;
19     }
```

1.5.12 遍历表

1) 函数基本说明

函数名称是 ListTraverse(L,visit()), 初始条件是线性表 L 已存在; 操作结果是依次对 L 的每个数据元素调用函数 visit()(12)。

Listing 12 目标程序

```
1     status ListTraverse(SqList L)。
2     {
3         if(L.elem==NULL)
4         {
5             return INFEASIBLE;
6         }
7         for(int i=0;i<L.length;i++)
8         {
9             printf("%d ",L.elem[i]);
10        }
11        return OK;
12    }
```

1.5.13 最大连续子数组和

1) 函数基本说明

函数名称是 MaxSubArray(L); 初始条件是线性表 L 已存在且非空, 请找出一个具有最大和的连续子数组 (子数组最少包含一个元素), 操作结果是其最大和 (13);

Listing 13 目标程序

```
1     int maxSubArraySum(SqList L) {
2         int max = L.elem[0];
3         for (int i = 0; i < L.length; i++) {
4             int sum = 0;
5             for (int j = i; j < L.length; j++) {
```

```
6             sum += L.elem[j];
7             if (sum > max) {
8                 max = sum;
9             }
10        }
11    }
12    return max;
13 }
```

1.5.14 和为 K 的子数组

1) 函数基本说明

函数名称是 SubArrayNum(L,k); 初始条件是线性表 L 已存在且非空, 操作结果是该数组中和为 k 的连续子数组的个数 (14);

Listing 14 目标程序

```
1     int SubArrayNum(SqList L, int k) {
2         int count = 0;
3         for (int i = 0; i < L.length; i++) {
4             int sum = 0;
5             for (int j = i; j < L.length; j++) {
6                 sum += L.elem[j];
7                 if (sum == k) {
8                     count++;
9                 }
10            }
11        }
12        return count;
13    }
```

1.5.15 顺序表排序

1) 函数基本说明

函数名称是 sortList(L); 初始条件是线性表 L 已存在; 操作结果是将 L 由小到大排序 (15);

Listing 15 目标程序

```
1     void sortList(SqList& L) {
```

```
2         std::sort(L.elem, L.elem + L.length);
3     }
```

1.5.16 实现线性表的文件形式保存

1) 函数基本说明

其中, □ 需要设计文件数据记录格式, 以高效保存线性表数据逻辑结构 (D,R) 的完整信息; □ 需要设计线性表文件保存和加载操作合理模式。附录 B 提供了文件存取的参考方法 (16)。

Listing 16 目标程序

```
1     status SaveList(SqList L, char FileName[])
2     {
3         if(L.elem==NULL) return INFEASIBLE;
4         FILE *w=fopen(FileName, "w");
5         for(int i=0; i<L.length; i++){
6             fprintf(w, "%d ", L.elem[i]);
7         }
8         fclose(w);
9     }
10    status LoadList(SqList &L, char FileName[])
11    {
12        int t;
13        if(L.elem!=NULL) return INFEASIBLE;
14        L.elem=(ElemType *)malloc(sizeof(ElemType)*LIST_INIT_SIZE);
15        L.length=0;
16        FILE *r=fopen(FileName, "r");
17        while(fscanf(r, "%d", &t) != EOF){
18            L.elem[L.length++]=t;
19        }
20        fclose(r);
21    }
```

1.5.17 实现多个线性表管理

1) 函数基本说明

设计相应的数据结构管理多个线性表的查找、添加、移除等功能 (17)。

Listing 17 目标程序

```
1  status AddList(LISTS &Lists, char ListName[])
2  {
3      InitList(Lists.elem[Lists.length].L);
4      strcpy(Lists.elem[Lists.length].name, ListName);
5      Lists.length++;
6      return OK;
7  }
8
9  status RemoveList(LISTS &Lists, char ListName[])
10 {
11     for(int i = 0; i < Lists.length; i++)
12     {
13         if(strcmp(ListName, Lists.elem[i].name) == 0)
14         {
15             DestroyList(Lists.elem[i].L);
16             for(int j = i; j < Lists.length - 1; j++)
17             {
18                 Lists.elem[j] = Lists.elem[j + 1];
19             }
20             Lists.length--;
21             return OK;
22         }
23     }
24     return ERROR;
25 }
26
27 int LocateList(LISTS Lists, char ListName[])
28 {
29     for(int i = 0; i < Lists.length; i++)
30     {
31         if(strcmp(ListName, Lists.elem[i].name) == 0)
32         {
33             count = i + 1;
34             // 复制 elem[i].L 到 L
35             if (L.elem != NULL) {
36                 DestroyList(L);
37             }
38             InitList(L);
39             for (int j = 0; j < Lists.elem[i].L.length; j++) {
40                 ElemType e;
```



```
41         GetElem(Lists.elem[i].L, j + 1, e);
42         ListInsert(L, j + 1, e);
43     }
44     return i + 1;
45 }
46 }
47 return 0;
48 }
```

1.6 系统测试

1.6.1 测试说明

为测试各函数功能是否完整，现计划一组数据去进行测试，计划数据如下

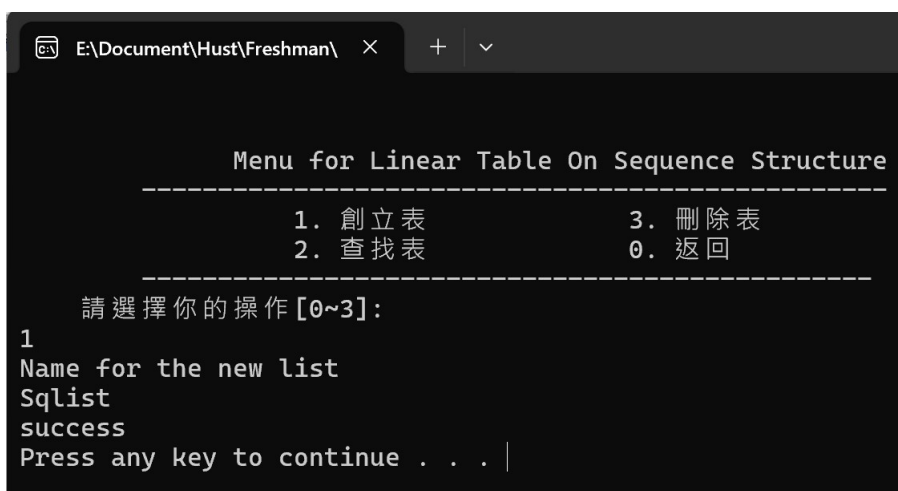
1. 5
2. 3
3. 4
4. 2

然后我会先测试好多表操作再进行后面的测试

1.6.2 实际测试

1) 建立表

如图所示我们成功地建立了一个名为”Sqlist”的顺序表

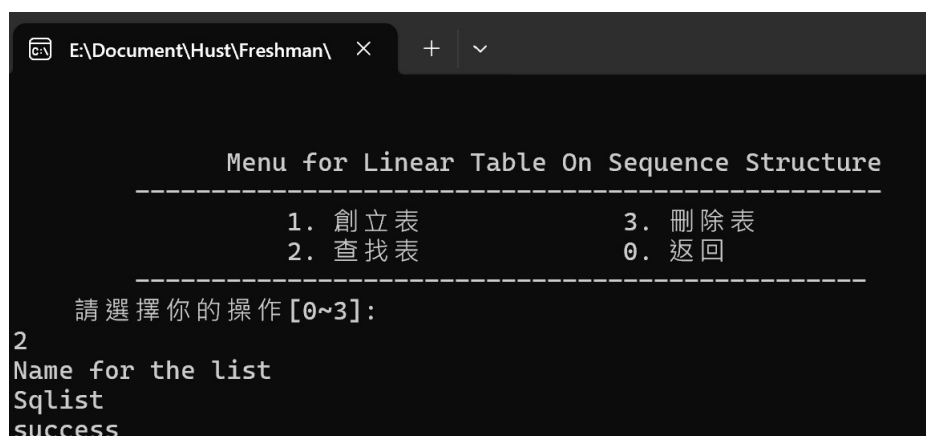


```
E:\Document\Hust\Freshman\ × + v
Menu for Linear Table On Sequence Structure
-----
1. 创立表          3. 删除表
2. 查找表          0. 返回
-----
請選擇你的操作[0~3]:
1
Name for the new list
Sqlist
success
Press any key to continue . . . |
```

图 1-1 建立表

2) 查找表

如图所示我们成功地找到了刚刚建立的表”Sqlist”



```
E:\Document\Hust\Freshman\ >
Menu for Linear Table On Sequence Structure
-----
1. 創立表          3. 刪除表
2. 查找表          0. 返回
-----
請選擇你的操作[0~3]:
2
Name for the list
Sqlist
success
```

图 1-2 查找表

3) 删除表

如图所示这里删除失败是因为我们并没有建立名为”abc” 的表因此此表不存在找不到表所以删除失败。



```
E:\Document\Hust\Freshman\ >
Menu for Linear Table On Sequence Structure
-----
1. 創立表          3. 刪除表
2. 查找表          0. 返回
-----
請選擇你的操作[0~3]:
3
Delete the List:(input list's name)
abc
刪除失敗
```

图 1-3 删除表

4) 初始化表

这里因为多表管理已经建立表 Sqlist 了，因此 initlist 时表已存在

```
Menu for Linear Table On Sequence Structure
-----
1. InitList          7. LocateElem
2. DestroyList       8. PriorElem
3. ClearList         9. NextElem
4. ListEmpty         10. ListInsert
5. ListLength        11. ListDelete
6. GetElem           12. ListTraverse
                     13. MaxSubArray
                     14. SubArrayNum
                     15. sortList
                     16. Savelist
                     17. LoadFile
                     18. WithMultiLists
                     0. turn back
-----
請選擇你的操作[0~18]:1
線性表已存在！
Press any key to continue . . .
```

图 1-4 初始化表

5) 判空表

这里我们判断表是否为空，目前因为没有输入任何的数据内容因此表为空

```
Menu for Linear Table On Sequence Structure
-----
1. InitList          7. LocateElem
2. DestroyList       8. PriorElem
3. ClearList         9. NextElem
4. ListEmpty         10. ListInsert
5. ListLength        11. ListDelete
6. GetElem           12. ListTraverse
                     13. MaxSubArray
                     14. SubArrayNum
                     15. sortList
                     16. Savelist
                     17. LoadFile
                     18. WithMultiLists
                     0. turn back
-----
請選擇你的操作[0~18]:4
線性表為空！
Press any key to continue . . . |
```

图 1-5 判空表

6) 插入元素

这里我们在表中插入一个元素 5

```
Menu for Linear Table On Sequence Structure
-----
1. InitList          7. LocateElem
2. DestroyList       8. PriorElem
3. ClearList         9. NextElem
4. ListEmpty         10. ListInsert
5. ListLength        11. ListDelete
6. GetElem           12. ListTraverse
                     13. MaxSubArray
                     14. SubArrayNum
                     15. sortList
                     16. Savelist
                     17. LoadFile
                     18. WithMultiLists
                     0. turn back
-----
請選擇你的操作[0~18]:10
輸入在第i個元素前插入的元素:1 5
OK
Press any key to continue . . . |
```

图 1-6 插入元素

7) 查表长

因为目前表中只有元素 5，一个元素因此长度为 1

```
Menu for Linear Table On Sequence Structure
-----
1. InitList          7. LocateElem
2. DestroyList       8. PriorElem
3. ClearList         9. NextElem
4. ListEmpty         10. ListInsert
5. ListLength        11. ListDelete
6. GetElem           12. ListTraverse
                     13. MaxSubArray
                     14. SubArrayNum
                     15. sortList
                     16. Savelist
                     17. LoadFile
                     18. WithMultiLists
                     0. turn back
-----
請選擇你的操作[0~18]:5
線性表長度為1
Press any key to continue . . . |
```

图 1-7 查表长

8) 找特定元素

```
Menu for Linear Table On Sequence Structure
-----
1. InitList          7. LocateElem
2. DestroyList       8. PriorElem
3. ClearList         9. NextElem
4. ListEmpty         10. ListInsert
5. ListLength        11. ListDelete
6. GetElem           12. ListTraverse
                    13. MaxSubArray
                    14. SubArrayNum
                    15. sortList
                    16. Savelist
                    17. LoadFile
                    18. WithMultiLists
                    0. turn back
-----
請選擇你的操作[0~18]:7
輸入你要查找的元素:3
查找失敗!
Press any key to continue . . . |
```

图 1-8 找特定元素

9) 遍历表

只里我们插入元素 5 3 4 2，然后遍历一下看看能否顺利输出

```
Menu for Linear Table On Sequence Structure
-----
1. InitList          7. LocateElem
2. DestroyList       8. PriorElem
3. ClearList         9. NextElem
4. ListEmpty         10. ListInsert
5. ListLength        11. ListDelete
6. GetElem           12. ListTraverse
                    13. MaxSubArray
                    14. SubArrayNum
                    15. sortList
                    16. Savelist
                    17. LoadFile
                    18. WithMultiLists
                    0. turn back
-----
請選擇你的操作[0~18]:12
5 3 4 2
Press any key to continue . . . |
```

图 1-9 遍历表

10) 查找前驱

这里数列仍为 5 3 4 2，我们找 3 的前驱看不是 5。

```
Menu for Linear Table On Sequence Structure
-----
1. InitList          7. LocateElem
2. DestroyList       8. PriorElem
3. ClearList         9. NextElem
4. ListEmpty         10. ListInsert
5. ListLength        11. ListDelete
6. GetElem           12. ListTrabverse

13. MaxSubArray      14. SubArrayNum
15. sortList         16. Savelist
17. LoadFile         18. WithMultiLists
0. turn back
-----
請選擇你的操作[0~18]:8
輸入你要查找的元素前驅:3
3的前驅為5
Press any key to continue . . . |
```

图 1-10 查找前驱

11) 查找后驱

这里数列仍为 5 3 4 2，我们找 3 的后驱看不是 4。

```
Menu for Linear Table On Sequence Structure
-----
1. InitList          7. LocateElem
2. DestroyList       8. PriorElem
3. ClearList         9. NextElem
4. ListEmpty         10. ListInsert
5. ListLength        11. ListDelete
6. GetElem           12. ListTrabverse

13. MaxSubArray      14. SubArrayNum
15. sortList         16. Savelist
17. LoadFile         18. WithMultiLists
0. turn back
-----
請選擇你的操作[0~18]:9
輸入你要查找的元素後繼:3
3的後繼為4
Press any key to continue . . . |
```

图 1-11 查找后驱

12) 删除元素

这里我们试一下删除 5 号位的元素，但我们知道 5 号位没有元素因为不会成功删除

```
Menu for Linear Table On Sequence Structure
-----
1. InitList          7. LocateElem
2. DestroyList       8. PriorElem
3. ClearList         9. NextElem
4. ListEmpty         10. ListInsert
5. ListLength        11. ListDelete
6. GetElem           12. ListTraverse
                     13. MaxSubArray
                     14. SubArrayNum
                     15. sortList
                     16. Savelist
                     17. LoadFile
                     18. WithMultiLists
                     0. turn back
-----
請選擇你的操作[0~18]:11
輸入你要刪除的第i個元素:5
查找失敗！
```

图 1-12 初始化表

13) 排序表

我们把数列 5 3 4 2 排序一下看是不是顺利排成 2 3 4 5

```
Menu for Linear Table On Sequence Structure
-----
1. InitList          7. LocateElem
2. DestroyList       8. PriorElem
3. ClearList         9. NextElem
4. ListEmpty         10. ListInsert
5. ListLength        11. ListDelete
6. GetElem           12. ListTraverse
                     13. MaxSubArray
                     14. SubArrayNum
                     15. sortList
                     16. Savelist
                     17. LoadFile
                     18. WithMultiLists
                     0. turn back
-----
請選擇你的操作[0~18]:12
2 3 4 5
Press any key to continue . . . |
```

图 1-13 排序表

14) 存取表

这里我们先保存为 sqlist1.txt 然后清空表再加载一下看能不能找回来，我们发现是可以加载回来的啊，仍然是 2 3 4 5

```
Menu for Linear Table On Sequence Structure
-----
1. InitList          7. LocateElem
2. DestroyList       8. PriorElem
3. ClearList         9. NextElem
4. ListEmpty         10. ListInsert
5. ListLength        11. ListDelete
6. GetElem           12. ListTraverse

13. MaxSubArray      14. SubArrayNum
15. sortList         16. Savelist
17. LoadFile         18. WithMultiLists
0. turn back
-----
請選擇你的操作[0~18]:16
輸入保存文件名sqlist1
```

图 1-14 保存表

```
Menu for Linear Table On Sequence Structure
-----
1. InitList          7. LocateElem
2. DestroyList       8. PriorElem
3. ClearList         9. NextElem
4. ListEmpty         10. ListInsert
5. ListLength        11. ListDelete
6. GetElem           12. ListTraverse

13. MaxSubArray      14. SubArrayNum
15. sortList         16. Savelist
17. LoadFile         18. WithMultiLists
0. turn back
-----
請選擇你的操作[0~18]:17
輸入加載文件名sqlist1
Press any key to continue . . . |
```

图 1-15 加载表


```
Menu for Linear Table On Sequence Structure
-----
1. InitList          7. LocateElem
2. DestroyList       8. PriorElem
3. ClearList         9. NextElem
4. ListEmpty         10. ListInsert
5. ListLength        11. ListDelete
6. GetElem           12. ListTraverse
                     13. MaxSubArray
                     14. SubArrayNum
                     15. sortList
                     16. Savelist
                     17. LoadFile
                     18. WithMultiLists
                     0. turn back
-----
請選擇你的操作[0~18]:12
2 3 4 5
Press any key to continue . . . |
```

图 1-16 遍历表

15) 最大连续子数组和

我们看看最大连续子数组和是不是 $2+3+4+5=14$

```
Menu for Linear Table On Sequence Structure
-----
1. InitList          7. LocateElem
2. DestroyList       8. PriorElem
3. ClearList         9. NextElem
4. ListEmpty         10. ListInsert
5. ListLength        11. ListDelete
6. GetElem           12. ListTraverse
                     13. MaxSubArray
                     14. SubArrayNum
                     15. sortList
                     16. Savelist
                     17. LoadFile
                     18. WithMultiLists
                     0. turn back
-----
請選擇你的操作[0~18]:13
最大子數組和是14Press any key to continue . . . |
```

图 1-17 最大连续子数组和

16) 和为 K 的子数组

我们输入 7 看看是不是能够找到 $3+4=7$ 这对数组

```
Menu for Linear Table On Sequence Structure
-----
1. InitList          7. LocateElem
2. DestroyList       8. PriorElem
3. ClearList         9. NextElem
4. ListEmpty         10. ListInsert
5. ListLength        11. ListDelete
6. GetElem           12. ListTraverse
                    13. MaxSubArray
                    14. SubArrayNum
                    15. sortList
                    16. Savelist
                    17. LoadFile
                    18. WithMultiLists
                    0. turn back
-----
請選擇你的操作[0~18]:14
設定子數組和7
有1個Press any key to continue . . . |
```

图 1-18 和为 K 的子数组

至此，我们可以说是成功测试了程序的基本要求了，实现了顺序表的多表管理和基本运算要求。

1.7 实验小结

在本次 C 语言实现顺序表多表管理的实验中，我成功完成了对多个顺序表的创建、插入、删除、查找及遍历等核心操作。通过将理论知识转化为实践代码，我不仅加深了对顺序表数据结构特性的理解，也掌握了如何通过动态内存分配灵活管理多个顺序表的存储与操作。

在实验过程中，我采用结构体数组存储多个顺序表的表头信息，每个表头结构体包含了顺序表的容量、当前长度以及数据元素数组指针。这种设计方式有效实现了对多个顺序表的统一管理，同时也通过模块化编程，将每个功能封装为独立函数，增强了代码的可读性与可维护性。例如，在插入元素的函数实现中，我充分考虑了顺序表满时的动态扩容操作，确保了数据插入的可靠性；而在删除元素函数里，则通过合理的元素移动策略，维持了顺序表数据的连续性。

2 基于邻接表的图实现

3 实验目的

通过实验达到：

1. 加深对图的概念、基本运算的理解
2. 熟练掌握图的逻辑结构与物理结构的关系
3. 以邻接表作为物理结构，熟练掌握图基本运算的实现。

4 基于顺序存储结构的线性表实现

1. 创建图：函数名称是 `CreateGraph(G,V,VR)`；初始条件是 V 是图的顶点集， VR 是图的关系集；操作结果是按 V 和 VR 的定义构造图 G ；注：□ 要求图 G 中顶点关键字具有唯一性。后面各操作的实现，也都要满足一个图中关键字的唯一性，不再赘述；□ V 和 VR 对应的是图的逻辑定义形式，比如 V 为顶点序列， VR 为关键字对的序列。不能将邻接矩阵等物理结构来代替 V 和 VR 。
2. 销毁图：函数名称是 `DestroyGraph(G)`；初始条件图 G 已存在；操作结果是销毁图 G ；
3. 查找顶点：函数名称是 `LocateVex(G,u)`；初始条件是图 G 存在， u 是和 G 中顶点关键字类型相同的给定值；操作结果是若 u 在图 G 中存在，返回关键字为 u 的顶点位置序号（简称位序），否则返回其它表示“不存在”的信息；
4. 顶点赋值：函数名称是 `PutVex (G,u,value)`；初始条件是图 G 存在， u 是和 G 中顶点关键字类型相同的给定值；操作结果是对关键字为 u 的顶点赋值 $value$ ；
5. 获得第一邻接点：函数名称是 `FirstAdjVex(G, u)`；初始条件是图 G 存在， u 是 G 中顶点的位序；操作结果是返回 u 对应顶点的第一个邻接顶点位序，如果 u 的顶点没有邻接顶点，否则返回其它表示“不存在”的信息；
6. 获得下一邻接点：函数名称是 `NextAdjVex(G, v, w)`；初始条件是图 G 存在， v 和 w 是 G 中两个顶点的位序， v 对应 G 的一个顶点， w 对应 v 的邻接顶点；操作结果是返回 v 的（相对于 w ）下一个邻接顶点的位序，如果 w 是最后一个邻接顶点，返回其它表示“不存在”的信息；
7. 插入顶点：函数名称是 `InsertVex(G,v)`；初始条件是图 G 存在， v 和 G 中的顶点具有相同特征；操作结果是在图 G 中增加新顶点 v 。（在这里也保持顶点关键字的唯一性）；
8. 删除顶点：函数名称是 `DeleteVex(G,v)`；初始条件是图 G 存在， v 是和 G 中顶点关键字类型相同的给定值；操作结果是在图 G 中删除关键字 v 对应的顶点以及相关的弧；
9. 插入弧：函数名称是 `InsertArc(G,v,w)`；初始条件是图 G 存在， v 、 w 是和 G 中顶点关键字类型相同的给定值；操作结果是在图 G 中增加弧 $\langle v,w \rangle$ ，如果

图 G 是无向图，还需要增加 $\langle w, v \rangle$;

10. 删除弧：函数名称是 `DeleteArc(G,v,w)`；初始条件是图 G 存在， v 、 w 是和 G 中顶点关键字类型相同的给定值；操作结果是在图 G 中删除弧 $\langle v, w \rangle$ ，如果图 G 是无向图，还需要删除 $\langle w, v \rangle$;
11. 深度优先搜索遍历：函数名称是 `DFS_Traverse(G,visit())`；初始条件是图 G 存在；操作结果是图 G 进行深度优先搜索遍历，依次对图中的每一个顶点使用函数 `visit` 访问一次，且仅访问一次；
12. 广度优先搜索遍历：函数名称是 `BFS_Traverse(G,visit())`；初始条件是图 G 存在；操作结果是图 G 进行广度优先搜索遍历，依次对图中的每一个顶点使用函数 `visit` 访问一次，且仅访问一次。

1. 附加功能：
2. 距离小于 k 的顶点集合：函数名称是 `VerticesSetLessThanK(G,v,k)`，初始条件是图 G 存在；操作结果是返回与顶点 v 距离小于 k 的顶点集合；
3. 顶点间最短路径和长度：函数名称是 `ShortestPathLength(G,v,w)`；初始条件是图 G 存在；操作结果是返回顶点 v 与顶点 w 的最短路径的长度；
4. 图的连通分量：函数名称是 `ConnectedComponentsNums(G)`，初始条件是图 G 存在；操作结果是返回图 G 的所有连通分量的个数；
5. 实现图的文件形式保存：其中，□ 需要设计文件数据记录格式，以高效保存图的数据逻辑结构 (D,R) 的完整信息；□ 需要设计图文件保存和加载操作合理模式。附录 B 提供了文件存取的方法；
6. 实现多个图管理：设计相应的数据结构管理多个图的查找、添加、移除等功能。

* 注：附加功能（5）实现多个图管理应能创建、添加、移除多个图，单图和多图的区别仅仅在于图管理的个数不同，多图管理应可自由切换到管理的每个图，并可单独对某图进行单图的所有操作。

4.1 问题描述

图的集合 G 是由集合 V 和集合 E 组成，即 $G=V,E$ ， V 表示图中所有顶点的集合， E 表示顶点之间所有边的集合。

图是一种数据结构，加上一组基本操作，就成了抽象数据类型。依据最小完备性和常用性相结合的原则，以函数形式定义了创建图、销毁图、查找顶点、获得顶

点值和顶点赋值等 12 种基本运算。即，我们利用邻接表的形式，完成对图的数据结构的实现。而在本次实验中，我们假定设计的图皆为无向图。

4.2 系统设计

首先为在 C 语言下实现上述功能，我们大致可以分为三个部分，主程序框架、多图管理框架菜单、对当前图操作框架菜单

主程序框架负责实现多图管理和对当前图操作的框架切换和退出终止的操作。

而多图管理则是负责多个图的读写操作，那么则需是编号或命名等操作去区分不同的图，因此我们需要设计一个 struct 结构去装下这些信息。

而最后一个对当前图操作则是对传入的图去进行图的基本运算等操作，负责装下这些函数和选择调用。

4.3 系统实现

在设计这么一个菜单框架时我们可以通过多种的方法去实现，那么我的方法是通过 switch() 函数和 goto 语法实现在多个 switch-while 框架中的切换，除此之外我们也可以通过函数指针去实现此菜单功能，然后利用 system("PAUSE"); 实现了消除了每次操作后的「清屏」，使得菜单功能选项仍然出现在屏上不需改变位置。

随后的则是建立多表操作如建立表，删除表，查找并选择表，返回主菜单判断是要继续对表操作还是终止操作，即对此表操作。

而在当前表操作时则是在加入多个基本运算之外还要加一个返回主菜单操作。

4.3.1 创建图

1) 函数基本说明

函数名称是 CreateCraph(G,V,V0,9R); 初始条件是 V 是图的顶点集，VR 是图的关系集；操作结果是按 V 和 VR 的定义构造图 G；

注：□ 要求图 G 中顶点关键字具有唯一性。后面各操作的实现，也都要满足一个图中关键字的唯一性，不再赘述；□ V 和 VR 对应的是图的逻辑定义形式，比

如 V 为顶点序列, VR 为关键字对的序列。不能将邻接矩阵等物理结构来代替 V 和 VR 。(18);

Listing 18 目标程序

```
1      #include<map>
2      std::map<int,int> flag;
3      int LocateVex(ALGraph &G, KeyType key) {
4          for (int i = 0; i < G.vexnum; ++i) {
5              if (G.vertices[i].data.key == key)
6                  return i;
7          }
8          return -1;
9      }
10     status CreateCraph(ALGraph &G,VertexType V[],KeyType VR[][2])
11     {
12         G.vexnum = 0;
13         while (G.vexnum <= MAX_VERTEX_NUM && V[G.vexnum].key != -1) { //
            假设以-1结束
14             G.vexnum++;
15         }
16         if (G.vexnum > MAX_VERTEX_NUM or G.vexnum==0) return ERROR;
17         for (int i = 0; i < G.vexnum; ++i) {
18             for (int j = i + 1; j < G.vexnum; ++j) {
19                 if (V[i].key == V[j].key)
20                     return ERROR;
21             }
22         }
23         for (int i = 0; i < G.vexnum; ++i) {
24             G.vertices[i].data = V[i];
25             G.vertices[i].firstarc = NULL;
26         }
27         G.arcnum = 0;
28         while (VR[G.arcnum][0] != -1 && VR[G.arcnum][1] != -1) { // 假设以-1结束
29             G.arcnum++;
30         }
31         for (int k = 0; k < G.arcnum; ++k) {
32             KeyType u = VR[k][0];
33             KeyType v = VR[k][1];
34
35             int i = LocateVex(G, u);
36             int j = LocateVex(G, v);
```

```
37         if (i == -1 || j == -1)
38             return ERROR;
39
40         // 将j添加到i的邻接表中
41         ArcNode *arc_i = (ArcNode *)malloc(sizeof(ArcNode));
42         if (!arc_i) return OVERFLOW;
43         arc_i->adjvex = j;
44         arc_i->nextarc = G.vertices[i].firstarc;
45         G.vertices[i].firstarc = arc_i;
46
47         // 将i添加到j的邻接表中
48         ArcNode *arc_j = (ArcNode *)malloc(sizeof(ArcNode));
49         if (!arc_j) {
50             free(arc_i);
51             return OVERFLOW;
52         }
53         arc_j->adjvex = i;
54         arc_j->nextarc = G.vertices[j].firstarc;
55         G.vertices[j].firstarc = arc_j;
56     }
57     G.kind = UDG;
58     return OK;
59 }
```

4.3.2 销毁图

1) 函数基本说明

函数名称是 DestroyGraph(G); 初始条件图 G 已存在; 操作结果是销毁图 G;
(19);

Listing 19 目标程序

```
1     status DestroyGraph(ALGraph &G)
2     {
3         for (int i = 0; i < G.vexnum; i++)
4         {
5             ArcNode *p = G.vertices[i].firstarc;
6             while (p)
7             {
8                 ArcNode *q = p;
9                 p = p->nextarc;
```



```
10         free(q);
11     }
12     G.vertices[i].firstarc = NULL;
13 }
14 G.vexnum = 0;
15 G.arcnum = 0;
16 return OK;
17 }
```

4.3.3 查找顶点

1) 函数基本说明

函数名称是 LocateVex(G,u); 初始条件是图 G 存在, u 是和 G 中顶点关键字类型相同的给定值; 操作结果是若 u 在图 G 中存在, 返回关键字为 u 的顶点位置序号 (简称位序), 否则返回其它表示“不存在”的信息; (20);

Listing 20 目标程序

```
1 int LocateVex(ALGraph G,KeyType u)
2
3 for (int i = 0; i < G.vexnum; ++i) {
4     if(G.vertices[i].data.key == u) return i;
5 }
6 return -1;
7 }
```

4.3.4 顶点赋值

1) 函数基本说明

函数名称是 PutVex (G,u,value); 初始条件是图 G 存在, u 是和 G 中顶点关键字类型相同的给定值; 操作结果是对关键字为 u 的顶点赋值 value; (21);

Listing 21 目标程序

```
1 int check(ALGraph G,int x,VertexType v){
2     for (int i = 0; i < G.vexnum; ++i) {
3         if(i == x) continue ;
4         if(v.key == G.vertices[i].data.key) return 0;
5     }
6     return 1;
7 }
```

```
7 }
8 status PutVex(ALGraph &G,KeyType u,VertexType value)
9 {
10     for (int i = 0; i < G.vexnum; ++i) {
11         if(G.vertices[i].data.key == u){
12             if(check(G,i,value)== 1){
13                 G.vertices[i].data=value;
14                 return OK;
15             }
16             else return ERROR;
17         }
18     }
19     return ERROR;
20 }
```

4.3.5 获得第一邻接点

1) 函数基本说明

函数名称是 FirstAdjVex(G,u); 初始条件是图 G 存在, u 是 G 中顶点的位序; 操作结果是返回 u 对应顶点的第一个邻接顶点位序, 如果 u 的顶点没有邻接顶点, 否则返回其它表示“不存在”的信息; (22);

Listing 22 目标程序

```
1 int LocateVex(ALGraph G,KeyType u)
2 {
3     for (int i = 0; i < G.vexnum; ++i) {
4         if(G.vertices[i].data.key == u) return i;
5     }
6     return -1;
7 }
8 int FirstAdjVex(ALGraph G,KeyType u)
9 //根据u在图G中查找顶点, 查找成功返回顶点u的第一邻接顶点位序, 否则返回-1;
10 {
11     int t = LocateVex(G,u);
12     if(G.vertices[t].firstarc!=NULL) return G.vertices[t].firstarc->adjvex;
13     return -1;
14 }
```

4.3.6 获得下一邻接点

1) 函数基本说明

函数名称是 NextAdjVex(G, v, w); 初始条件是图 G 存在, v 和 w 是 G 中两个顶点的位序, v 对应 G 的一个顶点, w 对应 v 的邻接顶点; 操作结果是返回 v 的 (相对于 w) 下一个邻接顶点的位序, 如果 w 是最后一个邻接顶点, 返回其它表示 “不存在” 的信息; (23);

Listing 23 目标程序

```
1  int LocateVex(ALGraph G,KeyType u)
2  {
3      for (int i = 0; i < G.vexnum; ++i) {
4          if(G.vertices[i].data.key == u) return i;
5      }
6      return -1;
7  }
8  int NextAdjVex(ALGraph G,KeyType v,KeyType w)
9  {
10     int t = LocateVex(G,v);
11     int wt = LocateVex(G,w);
12     if (t==-1 or wt == -1) return -1;
13     ArcNode *p = G.vertices[t].firstarc;
14     while(p!=NULL){
15         if(p->adjvex==wt){
16             if(p->nextarc!=NULL) return p->nextarc->adjvex;
17             else return -1;
18         }
19         p=p->nextarc;
20     }
21     return -1;
22 }
```

4.3.7 插入顶点

1) 函数基本说明

函数名称是 InsertVex(G,v); 初始条件是图 G 存在, v 和 G 中的顶点具有相同特征; 操作结果是在图 G 中增加新顶点 v。(在这里也保持顶点关键字的唯一性) (24);

Listing 24 目标程序

```
1 status InsertVex(ALGraph &G,VertexType v)
2 {
3     if(G.vexnum == MAX_VERTEX_NUM) return ERROR;
4     for(int i=0;i<G.vexnum;i++){
5         if(G.vertices[i].data.key==v.key) return ERROR;
6     }
7     G.vertices[G.vexnum].data = v;
8     G.vertices[G.vexnum].firstarc=NULL;
9     G.vexnum++;
10    return OK;
11 }
```

4.3.8 删除顶点

1) 函数基本说明

函数名称是 DeleteVex(G,v); 初始条件是图 G 存在, v 是和 G 中顶点关键字类型相同的给定值; 操作结果是在图 G 中删除关键字 v 对应的顶点以及相关的弧; (25);

Listing 25 目标程序

```
1 int LocateVex(ALGraph G,KeyType u)
2 {
3     for (int i = 0; i < G.vexnum; ++i) {
4         if(G.vertices[i].data.key == u) return i;
5     }
6     return -1;
7 }
8 status DeleteVex(ALGraph &G,KeyType v)
9 {
10    if(G.vexnum == 1) return ERROR;
11    int pos_v = LocateVex(G, v);
12    if (pos_v == -1) return ERROR;
13
14    ArcNode *current = G.vertices[pos_v].firstarc;
15    while (current != NULL) {
16        int w_pos = current->adjvex;
17
18        ArcNode *w_prev = NULL;
```

```
19     ArcNode *w_p = G.vertices[w_pos].firstarc;
20     while (w_p != NULL) {
21         if (w_p->adjvex == pos_v) {
22             if (w_prev == NULL) {
23                 G.vertices[w_pos].firstarc = w_p->nextarc;
24             } else {
25                 w_prev->nextarc = w_p->nextarc;
26             }
27             ArcNode *temp = w_p;
28             w_p = w_p->nextarc;
29             free(temp);
30             G.arcnum--;
31             break;
32         } else {
33             w_prev = w_p;
34             w_p = w_p->nextarc;
35         }
36     }
37     ArcNode *temp = current;
38     current = current->nextarc;
39     free(temp);
40 }
41 G.vertices[pos_v].firstarc = NULL;
42
43 for (int i = pos_v; i < G.vexnum - 1; i++) {
44     G.vertices[i] = G.vertices[i + 1];
45 }
46 G.vexnum--;
47 for (int i = 0; i < G.vexnum; i++) {
48     ArcNode *p = G.vertices[i].firstarc;
49     while (p != NULL) {
50         if (p->adjvex > pos_v) {
51             p->adjvex--;
52         }
53         p = p->nextarc;
54     }
55 }
56 return OK;
57 }
```

4.3.9 插入弧

1) 函数基本说明

函数名称是 InsertArc(G,v,w); 初始条件是图 G 存在, v、w 是和 G 中顶点关键字类型相同的给定值; 操作结果是在图 G 中增加弧 $\langle v,w \rangle$, 如果图 G 是无向图, 还需要增加 $\langle w,v \rangle$; (26);

Listing 26 目标程序

```
1  int LocateVex(ALGraph G,KeyType u)
2  {
3      for (int i = 0; i < G.vexnum; ++i) {
4          if(G.vertices[i].data.key == u) return i;
5      }
6      return -1;
7  }
8  status InsertArc(ALGraph &G,KeyType v,KeyType w)
9  {
10     int vt = LocateVex(G,v),wt = LocateVex(G,w);
11     if(vt == -1 or wt == -1) return ERROR;
12     ArcNode *p = G.vertices[wt].firstarc;
13     while(p!=NULL){
14         if(p->adjvex == vt) return ERROR;
15         p=p->nextarc;
16     }
17     ArcNode *temp1 = (ArcNode *)malloc(sizeof(ArcNode));
18     temp1->nextarc = G.vertices[vt].firstarc;
19     temp1->adjvex = wt;
20     G.vertices[vt].firstarc = temp1;
21     ArcNode *temp2 = (ArcNode *)malloc(sizeof(ArcNode));
22     temp2->nextarc = G.vertices[wt].firstarc;
23     temp2->adjvex = vt;
24     G.vertices[wt].firstarc = temp2;
25     G.arcnum++;
26     return OK;
27 }
```

4.3.10 删除弧

1) 函数基本说明

函数名称是 DeleteArc(G,v,w); 初始条件是图 G 存在, v、w 是和 G 中顶点关键字类型相同的给定值; 操作结果是在图 G 中删除弧 $\langle v,w \rangle$, 如果图 G 是无向图, 还需要删除 $\langle w,v \rangle$; (27);

Listing 27 目标程序

```
1  int LocateVex(ALGraph G,KeyType u)
2  {
3      for (int i = 0; i < G.vexnum; ++i) {
4          if(G.vertices[i].data.key == u) return i;
5      }
6      return -1;
7  }
8  status DeleteArc(ALGraph &G,KeyType v,KeyType w)
9  {
10     int v_pos = LocateVex(G, v);
11     int w_pos = LocateVex(G, w);
12     if (v_pos == -1 || w_pos == -1) return ERROR;
13     ArcNode *prev = NULL;
14     ArcNode *curr = G.vertices[v_pos].firstarc;
15     bool found = false;
16     while (curr != NULL) {
17         if (curr->adjvex == w_pos) {
18             if (prev == NULL) {
19                 G.vertices[v_pos].firstarc = curr->nextarc;
20             } else {
21                 prev->nextarc = curr->nextarc;
22             }
23             free(curr);
24             found = true;
25             break;
26         }
27         prev = curr;
28         curr = curr->nextarc;
29     }
30     if (!found) return ERROR;
31     prev = NULL;
32     curr = G.vertices[w_pos].firstarc;
```

```
33     while (curr != NULL) {
34         if (curr->adjvex == v_pos) {
35             if (prev == NULL) {
36                 G.vertices[w_pos].firstarc = curr->nextarc;
37             } else {
38                 prev->nextarc = curr->nextarc;
39             }
40             free(curr);
41             break;
42         }
43         prev = curr;
44         curr = curr->nextarc;
45     }
46     G.arcnum--;
47     return OK;
48 }
```

4.3.11 深度优先搜索遍历

1) 函数基本说明

函数名称是 DFSTraverse(G,visit()); 初始条件是图 G 存在；操作结果是图 G 进行深度优先搜索遍历，依次对图中的每一个顶点使用函数 visit 访问一次，且仅访问一次；(28)；

Listing 28 目标程序

```
1  #include <map>
2  std::map<int,int> flagdfs;
3  void dfs(ALGraph G,int t,void (*visit)(VertexType)){
4      visit(G.vertices[t].data);
5      flagdfs[t]=1;
6      ArcNode *p = G.vertices[t].firstarc;
7      while(p!=NULL){
8          if(flagdfs[p->adjvex]==0) dfs(G,p->adjvex,visit);
9          p=p->nextarc;
10     }
11 }
12 status DFSTraverse(ALGraph &G,void (*visit)(VertexType))
13 {
14     for(int i=0;i<G.vexnum;i++){
```



```
15         if(flagdfs[i]==0) dfs(G,i,visit);
16     }
17 }
```

4.3.12 广度优先搜索遍历

1) 函数基本说明

函数名称是 `BFS_Traverse(G,visit())`；初始条件是图 `G` 存在；操作结果是图 `G` 进行广度优先搜索遍历，依次对图中的每一个顶点使用函数 `visit` 访问一次，且仅访问一次。(29)；

Listing 29 目标程序

```
1  #include <map>
2  #include <queue>
3  std::map<int,int> flagbfs;
4  std::queue<int> q;
5  status BFS_Traverse(ALGraph &G,void (*visit)(VertexType))
6  {
7      for(int i=0;i<G.vexnum;i++){
8          if(flagbfs[i]==0){
9              q.push(i);
10             flagbfs[i]=1;
11         }
12         while(!q.empty()){
13             int u=q.front();
14             q.pop();
15             visit(G.vertices[u].data);
16             ArcNode *p = G.vertices[u].firstarc;
17             while(p!=NULL){
18                 if(flagbfs[p->adjvex]==0){
19                     q.push(p->adjvex);
20                     flagbfs[p->adjvex]=1;
21                 }
22                 p=p->nextarc;
23             }
24         }
25     }
26 }
```

4.3.13 实现图的文件形式保存

1) 函数基本说明

其中，□ 需要设计文件数据记录格式，以高效保存图的数据逻辑结构 (D,R) 的完整信息；□ 需要设计图文件保存和加载操作合理模式。附录 B 提供了文件存取的方法；

注：□ 保存到文件后，可以直接加载文件生成图。(30)；

Listing 30 目标程序

```
1  #include<map>
2  #include <algorithm>
3  std::map<int,int> flagsave;
4  int LocateVex(ALGraph &G, KeyType key) {
5      for (int i = 0; i < G.vexnum; ++i) {
6          if (G.vertices[i].data.key == key)
7              return i;
8      }
9      return -1;
10 }
11 status CreateCraph0(ALGraph &G,VertexType V[],KeyType VR[][2])
12 {
13     G.vexnum = 0;
14     while (G.vexnum <= MAX_VERTEX_NUM && V[G.vexnum].key != -1) {
15         G.vexnum++;
16     }
17     if (G.vexnum > MAX_VERTEX_NUM or G.vexnum==0) return ERROR;
18     for (int i = 0; i < G.vexnum; ++i) {
19         for (int j = i + 1; j < G.vexnum; ++j) {
20             if (V[i].key == V[j].key)
21                 return ERROR;
22         }
23     }
24     for (int i = 0; i < G.vexnum; ++i) {
25         G.vertices[i].data = V[i];
26         G.vertices[i].firstarc = NULL;
27     }
28     G.arcnum = 0;
29     while (VR[G.arcnum][0] != -1 && VR[G.arcnum][1] != -1) {
30         G.arcnum++;
31     }
```

```
32     for (int k = 0; k < G.arcnum; ++k) {
33         KeyType u = VR[k][0];
34         KeyType v = VR[k][1];
35         int i = LocateVex(G, u);
36         int j = LocateVex(G, v);
37         if (i == -1 || j == -1)
38             return ERROR;
39         ArcNode *arc_i = (ArcNode *)malloc(sizeof(ArcNode));
40         if (!arc_i) return OVERFLOW;
41         arc_i->adjvex = j;
42         arc_i->nextarc = G.vertices[i].firstarc;
43         G.vertices[i].firstarc = arc_i;
44         ArcNode *arc_j = (ArcNode *)malloc(sizeof(ArcNode));
45         if (!arc_j) {
46             free(arc_i);
47             return OVERFLOW;
48         }
49         arc_j->adjvex = i;
50         arc_j->nextarc = G.vertices[j].firstarc;
51         G.vertices[j].firstarc = arc_j;
52     }
53     G.kind = UDG;
54     return OK;
55 }
56
57 status SaveGraph(ALGraph G, char FileName[])
58 {
59     FILE *fp = fopen(FileName, "w");
60     for(int i=0; i<G.vexnum; i++){
61         fprintf(fp, "%d %s ", G.vertices[i].data.key, G.vertices[i].data.others);
62     }
63     fprintf(fp, "%d %s ", -1, "nil");
64     for(int i=0; i<G.vexnum; i++){
65         flagsave[i]=1;
66         ArcNode *p = G.vertices[i].firstarc;
67         while(p!=NULL){
68             if(flagsave[p->adjvex] == 0){
69                 fprintf(fp, "%d %d ", G.vertices[i].data.key,
70                     G.vertices[p->adjvex].data.key);
71             }
72             p=p->nextarc;
```

```
72         }
73     }
74     fprintf(fp, "%d %d ", -1, -1);
75     fclose(fp);
76     return OK;
77 }
78 status LoadGraph(ALGraph &G, char FileName[])
79 {
80     FILE *fp = fopen(FileName, "r");
81     VertexType V[21];
82     KeyType VR[100][2];
83     int i=0;
84     do {
85         fscanf(fp, "%d%s", &V[i].key, V[i].others);
86     } while(V[i++].key!=-1);
87     i=0;
88     do {
89         fscanf(fp, "%d%d", &VR[i][0], &VR[i][1]);
90         if(VR[i][0]>VR[i][1]) std::swap(VR[i][0], VR[i][1]);
91     } while(VR[i++][0]!=-1);
92     i--;
93     for(int j=0; j<i-1; j++){
94         for(int r=0; r<i-1-j; r++){
95             if(VR[r][0] > VR[r+1][0] || (VR[r][0] == VR[r+1][0] && VR[r][1] >
96                 VR[r+1][1])){
97                 std::swap(VR[r], VR[r+1]);
98             }
99         }
100     }
101     if(CreateCraph0(G, V, VR)==OK) {
102         fclose(fp);
103         return OK;
104     }
105     else{
106         fclose(fp);
107         return ERROR;
108     }
109 }
```

4.3.14 距离小于 k 的顶点集合

1) 函数基本说明

函数名称是 VerticesSetLessThanK(G,v,k)，初始条件是图 G 存在；操作结果是返回与顶点 v 距离小于 k 的顶点集合；(31)；

Listing 31 目标程序

```
1 int VerticesSetLessThanK(ALGraph &G,int v,int k){
2     int u0= LocateVex(G,v),s[100]={-1};
3     if(u0==-1) return ERROR;
4     q.push(u0);
5     s[u0]=0;
6     flagbfs[u0]=1;
7     printf("%d %s\n",G.vertices[u0].data.key,G.vertices[u0].data.others);
8     while(!q.empty()){
9         int u=q.front();
10        q.pop();
11        ArcNode *p = G.vertices[u].firstarc;
12        while(p!=NULL){
13            if(flagbfs[p->adjvex]==0){
14                s[p->adjvex]=s[u]+1;
15                if(s[p->adjvex]<k){
16                    flagbfs[p->adjvex]=1;
17                    q.push(p->adjvex);
18                    printf("%d
19                        %s\n",G.vertices[p->adjvex].data.key,G.vertices[p->adjvex].data.others);
20                }
21                p=p->nextarc;
22            }
23        }
24        return OK;
25    }
```

4.3.15 顶点间最短路径和长度

1) 函数基本说明

函数名称是 ShortestPathLength(G,v,w); 初始条件是图 G 存在；操作结果是返回顶点 v 与顶点 w 的最短路径的长度；(32)；

Listing 32 目标程序

```
1  int ShortestPathLength(ALGraph &G,int v,int k){
2      int u0= LocateVex(G,v),s[100]={-1},v0=LocateVex(G,k);
3      if(u0==-1 or v0==-1) return -1;
4      q.push(u0);
5      s[u0]=0;
6      flagbfs[u0]=1;
7      while(!q.empty()){
8          int u=q.front();
9          q.pop();
10         ArcNode *p = G.vertices[u].firstarc;
11         while(p!=NULL){
12             if(flagbfs[p->adjvex]==0){
13                 s[p->adjvex]=s[u]+1;
14                 if(p->adjvex==v0){
15                     return s[p->adjvex];
16                 }
17                 flagbfs[p->adjvex]=1;
18                 q.push(p->adjvex);
19             }
20             p=p->nextarc;
21         }
22     }
23     return ERROR;
24 }
```

4.3.16 图的连通分量

1) 函数基本说明

函数名称是 ConnectedComponentsNums(G)，初始条件是图 G 存在；操作结果是返回图 G 的所有连通分量的个数；(33)；

Listing 33 目标程序

```
1  int ConnectedComponentsNums(ALGraph &G){
2      if(G.vexnum==0) return -1;
3      int s=0;
4      for(int i=0;i<G.vexnum;i++){
5          if(flagbfs[i]==0){
6              q.push(i);
```

```
7         flagbfs[i]=1;
8     }
9     else continue;
10    while(!q.empty()){
11        int u=q.front();
12        q.pop();
13        ArcNode *p = G.vertices[u].firstarc;
14        while(p!=NULL){
15            if(flagbfs[p->adjvex]==0){
16                q.push(p->adjvex);
17                flagbfs[p->adjvex]=1;
18            }
19            p=p->nextarc;
20        }
21    }
22    s++;
23 }
24 return s;
25 }
```

4.3.17 实现多个图管理

1) 函数基本说明

设计相应的数据结构管理多个图的查找、添加、移除等功能。(39);

Listing 34 目标程序

```
1 status AddList(LISTS &Lists,char ListName[])
2 {
3     Lists.elem[Lists.length].G = ALGraph(); // C++ 方式初始化
4     ALGraph &graph = Lists.elem[Lists.length].G;
5     graph.vexnum = 0;
6     graph.arcnum = 0;
7     graph.kind = UDG;
8     for (int i = 0; i < MAX_VERTEX_NUM; i++) {
9         graph.vertices[i].firstarc = NULL;
10    }
11    strcpy(Lists.elem[Lists.length].name,ListName);
12    Lists.length++;
13    return OK;
14 }
```

```
15 status RemoveList(LISTS &Lists, char ListName[])
16 {
17     for(int i=0; i<Lists.length; i++){
18         if(strcmp(ListName, Lists.elem[i].name)==0){
19             for(int j=i; j<Lists.length-1; j++){
20                 Lists.elem[j]=Lists.elem[j+1];
21             }
22             Lists.length--;
23             return OK;
24         }
25     }
26     return ERROR;
27 }
28 int LocateList(LISTS Lists, char ListName[])
29 {
30     for(int i=0; i<Lists.length; i++){
31         if(strcmp(Lists.elem[i].name, ListName)==0) return i+1;
32     }
33     return ERROR;
34 }
```

4.3.18 初始化表

1) 函数基本说明

函数名称是 InitList(L); 初始条件是线性表 L 不存在; 操作结果是构造一个空的线性表 (35);

Listing 35 目标程序

```
1 status InitList(SqList& L)
2 {
3     if(L.elem == NULL)
4     {
5         L.elem=(ElemType *) malloc(sizeof(ElemType)*LIST_INIT_SIZE);
6         L.length=0;
7         L.listsize=LIST_INIT_SIZE;
8         return OK;
9     }
10    else
11    {
12        return INFEASIBLE;
```


13 }

14 }

4.4 系统测试

4.4.1 测试说明

为测试各函数功能是否完整，现计划一组数据去进行测试，计划数据如下
5 线性表 8 集合 7 二叉树 6 无向图 -1 nil 5 6 5 7 6 7 7 8 -1 -1

然后我会先测试好多表操作再进行后面的测试

4.4.2 实际测试

1) 建立表

如图所示我们成功地建立了一个名为”Graph” 的顺序表

```
多圖管理！
-----
1. AddGraph      3. LocateGraph
2. DestroyGraph  4. 返回選擇頁面
0.Exit
-----
請選擇你的操作[0~4]:1
添加一個圖，請輸入圖名稱：Graph
OK!
Press any key to continue . . .
```

图 4-1 建立图

2) 查找图

如图所示我们成功地找到了刚刚建立的图”Graph”

```
多圖管理！
-----
1. AddGraph          3. LocateGraph
2. DestroyGraph      4. 返回選擇頁面
0.Exit
-----
請選擇你的操作[0~4]:3
查找一個圖，請輸入圖名稱：Graph
OK! 按1對其進行單圖操作,按其他按鍵返回|
```

图 4-2 查找图

3) 删除表

如图所示这里删除失败是因为我们并没有建立名为”abc”的表因此此表不存在找不到表所以删除失败。

```
多圖管理！
-----
1. AddGraph          3. LocateGraph
2. DestroyGraph      4. 返回選擇頁面
0.Exit
-----
請選擇你的操作[0~4]:2
刪除一個圖，請輸入圖名稱：abc
圖不存在！
Press any key to continue . . . |
```

图 4-3 删除图

4) 创建无向图

这里我们输入我们计划好的图，也就是 [5 线性表 8 集合 7 二叉树 6 无向图 -1 nil 5 6 5 7 6 7 7 8 -1 -1]

```
單圖管理！
-----
1. CreateCraph      7. InsertVex
2. DestroyGraph    8. DeleteVex
3. LocateVex       9. InsertArc
4. PutVex          10. DeleteArc
5. FirstAdjVex     11. DFSTraverse
6. NextAdjVex      12. BFSTraverse
13. VerticesSetLessThanK  14. ShortestPathLength
15. ConnectedComponentsNums  16. SaveGraph
17. LoadGraph      18. 返回多線性表操作
0. Exit
-----
請選擇你的操作[0~18]:1
請輸入定義序列:
5 线性表 8 集合 7 二叉树 6 无向图 -1 nil 5 6 5 7 6 7 7 8 -1 -1
OK
Press any key to continue . . . |
```

图 4-4 初始化表

5) 查找顶点

这里我们查找一下顶点，输入了关键词 5 找找看，而才成功找到线性表

```
單圖管理！
-----
1. CreateCraph      7. InsertVex
2. DestroyGraph    8. DeleteVex
3. LocateVex       9. InsertArc
4. PutVex          10. DeleteArc
5. FirstAdjVex     11. DFSTraverse
6. NextAdjVex      12. BFSTraverse
13. VerticesSetLessThanK  14. ShortestPathLength
15. ConnectedComponentsNums  16. SaveGraph
17. LoadGraph      18. 返回多線性表操作
0. Exit
-----
請選擇你的操作[0~18]:3
輸入查找的關鍵字：5
5 ?性表
Press any key to continue . . . |
```

图 4-5 查找顶点

6) 顶点赋值

这里我们输入 6 9 有向图看看能否成功，乱码只是输出字型问题不影响结果

```
單圖管理！
-----
1. CreateCraph      7. InsertVex
2. DestroyGraph    8. DeleteVex
3. LocateVex       9. InsertArc
4. PutVex          10. DeleteArc
5. FirstAdjVex     11. DFSTraverse
6. NextAdjVex      12. BFSTraverse
13. VerticesSetLessThanK  14. ShortestPathLength
15. ConnectedComponentsNums  16. SaveGraph
17. LoadGraph      18. 返回多線性表操作
0. Exit
-----
請選擇你的操作[0~18]:4
輸入關鍵字與修改為的值：6 9 有向圖
OK！
Press any key to continue . . . |
```

图 4-6 顶点赋值

7) 获得第一邻接点

这里我们输入关键词 7 看能否找到 8 集合

```
單圖管理！
-----
1. CreateCraph      7. InsertVex
2. DestroyGraph    8. DeleteVex
3. LocateVex       9. InsertArc
4. PutVex          10. DeleteArc
5. FirstAdjVex     11. DFSTraverse
6. NextAdjVex      12. BFSTraverse
13. VerticesSetLessThanK  14. ShortestPathLength
15. ConnectedComponentsNums  16. SaveGraph
17. LoadGraph      18. 返回多線性表操作
0. Exit
-----
請選擇你的操作[0~18]:5
輸入要查找的關鍵字：7
8 集合
Press any key to continue . . . |
```

图 4-7 获得第一邻接点

8) 获得下一邻接点

这里我们输入 7 8 看能否找到刚才顶点赋值 9 有向图

```
單圖管理！
-----
1. CreateCraph      7. InsertVex
2. DestroyGraph    8. DeleteVex
3. LocateVex       9. InsertArc
4. PutVex          10. DeleteArc
5. FirstAdjVex     11. DFSTraverse
6. NextAdjVex      12. BFSTraverse
13. VerticesSetLessThanK  14. ShortestPathLength
15. ConnectedComponentsNums 16. SaveGraph
17. LoadGraph      18. 返回多線性表操作
0. Exit
-----
請選擇你的操作[0~18]:6
輸入v和w: 7 8
9 有向圖
Press any key to continue . . . |
```

图 4-8 获得下一邻接点

9) 插入顶点

这里输入 6 无向图插入顶点

```
單圖管理！
-----
1. CreateCraph      7. InsertVex
2. DestroyGraph    8. DeleteVex
3. LocateVex       9. InsertArc
4. PutVex          10. DeleteArc
5. FirstAdjVex     11. DFSTraverse
6. NextAdjVex      12. BFSTraverse
13. VerticesSetLessThanK  14. ShortestPathLength
15. ConnectedComponentsNums 16. SaveGraph
17. LoadGraph      18. 返回多線性表操作
0. Exit
-----
請選擇你的操作[0~18]:7
輸入插入的點: 6 無向圖
OK!
```

图 4-9 插入顶点

10) 删除顶点

这里我们删除顶点 9 看能否成功删除

```
單圖管理！
-----
1. CreateCraph      7. InsertVex
2. DestroyGraph     8. DeleteVex
3. LocateVex        9. InsertArc
4. PutVex           10. DeleteArc
5. FirstAdjVex      11. DFSTraverse
6. NextAdjVex       12. BFSTraverse
13. VerticesSetLessThanK  14. ShortestPathLength
15. ConnectedComponentsNums  16. SaveGraph
17. LoadGraph       18. 返回多線性表操作
0. Exit
-----
請選擇你的操作[0~18]:8
輸入你要刪除的關鍵字：9
OK！
```

图 4-10 删除顶点

11) 深度优先搜索遍历

我们遍历一下 DFS 看看结果，乱码只是输出字型问题不影响结果

```
單圖管理！
-----
1. CreateCraph      7. InsertVex
2. DestroyGraph     8. DeleteVex
3. LocateVex        9. InsertArc
4. PutVex           10. DeleteArc
5. FirstAdjVex      11. DFSTraverse
6. NextAdjVex       12. BFSTraverse
13. VerticesSetLessThanK  14. ShortestPathLength
15. ConnectedComponentsNums  16. SaveGraph
17. LoadGraph       18. 返回多線性表操作
0. Exit
-----
請選擇你的操作[0~18]:11
5 ?性表 7 二叉? 8 集合
6 無向圖
```

图 4-11 深度优先搜索遍历

12) 广度优先搜索遍历

我们遍历一下 BFS 看看结果，乱码只是输出字型问题不影响结果

```
單圖管理！
-----
1. CreateCraph      7. InsertVex
2. DestroyGraph    8. DeleteVex
3. LocateVex       9. InsertArc
4. PutVex          10. DeleteArc
5. FirstAdjVex     11. DFSTraverse
6. NextAdjVex      12. BFSTraverse
13. VerticesSetLessThanK  14. ShortestPathLength
15. ConnectedComponentsNums  16. SaveGraph
17. LoadGraph      18. 返回多線性表操作
0. Exit
-----
請選擇你的操作[0~18]:12
5 ?性表 7 二叉? 8 集合
6 無向圖
Press any key to continue . . . |
```

图 4-12 广度优先搜索遍历

13) 文件保存与读入

这里我们存为 graph，然后再读入一次 graph。之后看下后面的操作是否能正常操作

```
單圖管理！
-----
1. CreateCraph      7. InsertVex
2. DestroyGraph    8. DeleteVex
3. LocateVex       9. InsertArc
4. PutVex          10. DeleteArc
5. FirstAdjVex     11. DFSTraverse
6. NextAdjVex      12. BFSTraverse
13. VerticesSetLessThanK  14. ShortestPathLength
15. ConnectedComponentsNums  16. SaveGraph
17. LoadGraph      18. 返回多線性表操作
0. Exit
-----
請選擇你的操作[0~18]:16
輸入檔案名：graph
OK
```

图 4-13 文件保存

```
單圖管理！
-----
1. CreateCraph      7. InsertVex
2. DestroyGraph    8. DeleteVex
3. LocateVex       9. InsertArc
4. PutVex          10. DeleteArc
5. FirstAdjVex     11. DFSTraverse
6. NextAdjVex      12. BFSTraverse
13. VerticesSetLessThanK  14. ShortestPathLength
15. ConnectedComponentsNums  16. SaveGraph
17. LoadGraph      18. 返回多線性表操作
0. Exit
-----
請選擇你的操作[0~18]:17
輸入檔案名：graph
OK
```

图 4-14 文件读入

14) 插入弧

这里我们插入一下 6 9 看下是否能看看插入失败

```
單圖管理！
-----
1. CreateCraph      7. InsertVex
2. DestroyGraph    8. DeleteVex
3. LocateVex       9. InsertArc
4. PutVex          10. DeleteArc
5. FirstAdjVex     11. DFSTraverse
6. NextAdjVex      12. BFSTraverse
13. VerticesSetLessThanK  14. ShortestPathLength
15. ConnectedComponentsNums  16. SaveGraph
17. LoadGraph      18. 返回多線性表操作
0. Exit
-----
請選擇你的操作[0~18]:7
輸入插入的點：6 9
ERROR！
```

图 4-15 插入弧

15) 删除弧

这里我们输入一下 6 9 看下是否能判断弧不存在


```
單圖管理！
-----
1. CreateCraph      7. InsertVex
2. DestroyGraph     8. DeleteVex
3. LocateVex        9. InsertArc
4. PutVex           10. DeleteArc
5. FirstAdjVex      11. DFSTraverse
6. NextAdjVex       12. BFSTraverse
13. VerticesSetLessThanK  14. ShortestPathLength
15. ConnectedComponentsNums 16. SaveGraph
17. LoadGraph       18. 返回多線性表操作
0. Exit
-----
請選擇你的操作[0~18]:10
輸入插入的弧<v,w>:6 9
插入失敗！
Press any key to continue . . . |
```

图 4-16 删除弧

16) 距离小于 k 的顶点集合

我们输入 5 8 看看能否找到这些顶点集

```
單圖管理！
-----
1. CreateCraph      7. InsertVex
2. DestroyGraph     8. DeleteVex
3. LocateVex        9. InsertArc
4. PutVex           10. DeleteArc
5. FirstAdjVex      11. DFSTraverse
6. NextAdjVex       12. BFSTraverse
13. VerticesSetLessThanK  14. ShortestPathLength
15. ConnectedComponentsNums 16. SaveGraph
17. LoadGraph       18. 返回多線性表操作
0. Exit
-----
請選擇你的操作[0~18]:13
輸入v與k: 5 8
5 ?性表
7 二叉?
8 集合
```

图 4-17 距离小于 k 的顶点集合

17) 顶点间最短路径和长度

我们看看 5 8 的最短路径是不是 2

```
單圖管理！
-----
1. CreateCraph      7. InsertVex
2. DestroyGraph     8. DeleteVex
3. LocateVex        9. InsertArc
4. PutVex           10. DeleteArc
5. FirstAdjVex      11. DFSTraverse
6. NextAdjVex       12. BFSTraverse
13. VerticesSetLessThanK  14. ShortestPathLength
15. ConnectedComponentsNums  16. SaveGraph
17. LoadGraph        18. 返回多線性表操作
0. Exit
-----
請選擇你的操作[0~18]:14
輸入v與w: 5 8
距離為2！
Press any key to continue . . . |
```

图 4-18 顶点间最短路径和长度

18) 图的连通分量

我们查看下能否成功输出 2

```
單圖管理！
-----
1. CreateCraph      7. InsertVex
2. DestroyGraph     8. DeleteVex
3. LocateVex        9. InsertArc
4. PutVex           10. DeleteArc
5. FirstAdjVex      11. DFSTraverse
6. NextAdjVex       12. BFSTraverse
13. VerticesSetLessThanK  14. ShortestPathLength
15. ConnectedComponentsNums  16. SaveGraph
17. LoadGraph        18. 返回多線性表操作
0. Exit
-----
請選擇你的操作[0~18]:15
连通分量有2個！
Press any key to continue . . . |
```

图 4-19 图的连通分量

4.5 实验小结

本次实验基于 C 语言实现了基于邻接表的图数据结构及其多图管理系统,核心目标是通过编程实践深入理解图的基本概念和操作。在基本实现方面,采用邻接表作为图的存储结构,通过 AdjList 数组存储顶点信息,每个顶点的邻接关系通过链表表示。通过这种结构,实现了图的基本操作,包括顶点的插入与删除、

边的添加与删除、图的遍历（深度优先和广度优先）等。在多图管理上，设计了 **GRAPHS** 结构体，通过数组管理多个图实例，并实现了图的添加、切换和删除功能。在实现过程中，我深刻体会到数据结构选择对程序性能的影响。邻接表在存储稀疏图时的优势明显，相较于邻接矩阵可以节省大量存储空间。同时，通过实践掌握了链表操作在图结构中的应用，包括节点的动态分配与释放、链表的遍历与修改等。在处理图的遍历操作时，理解了递归与队列在深度优先和广度优先算法中的不同应用场景。

参考文献

- [1] BAFNA V, PEVZNER P A. Genome Rearrangements and Sorting by Reversals[J/OL]. SIAM J. Comput., 1996, 25(2): 272–289.
<http://dx.doi.org/10.1137/S0097539793250627>.
- [2] SKINAZE. HUSTPaperTemp[EB/OL]. .
<https://github.com/skinaze/HUSTPaperTemp>.
- [3] MEHRABIAN A, RUSSELL J. An approach to environmental psychology[M]. [S.l.]: MIT, 1974.
- [4] REZAEI M, KLETTE R. Look at the driver, look at the road: No distraction! no accident![C] // CVPR. 2014: 129–136.
- [5] RAMNATH K, KOTERBA S, XIAO J, et al. Multi-view AMM fitting and construction[J]. International Journal of Computer Vision, 2008, 76: 183–204.
- [6] CHEN J, LI Z, JIN Y, et al. Video Saliency Prediction via Spatio-Temporal Reasoning[J]. Neurocomputing, 2021, 462: 59–68.
- [7] CHEN J, LI Q, LING H, et al. Audiovisual Saliency Prediction via Deep Learning[J]. Neurocomputing, 2021, 428: 248–258.
- [8] 尹圣君, 钱尚达, 李永代, et al. [M]. [S.l.]: 机械工业出版社, 2015.
- [9] ANON. GIEEE 802.21 Media Independent Handover (MIH)[S]. Washington University in St. Louis: IEEE, 2010.
- [10] 戴维民. [M]. [S.l.]: 学林出版社, 2008.
- [11] PRASAD N, KHOJASTEPOUR M A, JIANG M, et al. MU-MIMO: Demodulation at the Mobile Station[R]. 2009: 1–11.
- [12] PAULRAJ A J, Jr HEATH R W, SEBASTIAN P K, et al. Spatial Multiplexing in a Cellular Network[P]. 2000-5-23.
- [13] 立陶宛进入欧元时代 [N]. –. –.

附录 A 基于顺序存储结构线性表实现的源程序

Listing 36 目标程序

```
1  /* Linear Table On Sequence Structure */
2  #include <stdio.h>
3  #include <malloc.h>
4  #include <stdlib.h>
5  #include <string.h>
6  #include <algorithm>
7
8  /*-----page 10 on textbook -----*/
9  #define TRUE 1
10 #define FALSE 0
11 #define OK 1
12 #define ERROR 0
13 #define INFEASIBLE -1
14 #define OVERFLOW -2
15
16 typedef int status;
17 typedef int ElemType;
18
19 /*-----page 22 on textbook -----*/
20 #define LIST_INIT_SIZE 100
21 #define LISTINCREMENT 10
22
23 typedef struct{
24     ElemType * elem;
25     int length;
26     int listsize;
27 }SqList;
28
29 typedef struct{
30     struct {
31         char name[30];
32         SqList L;
33     } elem[10];
34     int length;
35 }LISTS;
36
37 LISTS Lists;
```

```
38 ElemType count = 0;
39 SqList L;
40
41 /*-----page 19 on textbook -----*/
42 status InitList(SqList& L);
43 status DestroyList(SqList& L);
44 status ClearList(SqList& L);
45 status ListEmpty(SqList L);
46 int ListLength(SqList L);
47 status GetElem(SqList L, int i, ElemType& e);
48 status LocateElem(SqList L, ElemType e);
49 status PriorElem(SqList L, ElemType cur, ElemType &pre_e);
50 status NextElem(SqList L, ElemType cur, ElemType& next_e);
51 status ListInsert(SqList &L, int i, ElemType e);
52 status ListDelete(SqList &L, int i, ElemType& e);
53 status ListTraverse(SqList L);
54
55 int maxSubArraySum(SqList L) {
56     int max = L.elem[0];
57     for (int i = 0; i < L.length; i++) {
58         int sum = 0;
59         for (int j = i; j < L.length; j++) {
60             sum += L.elem[j];
61             if (sum > max) {
62                 max = sum;
63             }
64         }
65     }
66     return max;
67 }
68
69 int SubArrayNum(SqList L, int k) {
70     int count = 0;
71     for (int i = 0; i < L.length; i++) {
72         int sum = 0;
73         for (int j = i; j < L.length; j++) {
74             sum += L.elem[j];
75             if (sum == k) {
76                 count++;
77             }
78         }
79     }
```

```
79     }
80     return count;
81 }
82
83 void sortList(SqlList& L) {
84     std::sort(L.elem, L.elem + L.length);
85 }
86
87 status SaveList(SqlList L, char FileName[])
88 {
89     if(L.elem == NULL) return INFEASIBLE;
90     FILE *w = fopen(FileName, "w");
91     if (w == NULL) return INFEASIBLE;
92     for(int i = 0; i < L.length; i++){
93         fprintf(w, "%d ", L.elem[i]);
94     }
95     fclose(w);
96     return OK;
97 }
98
99 status LoadList(SqlList &L, char FileName[])
100 {
101     int t;
102     if (L.elem != NULL) {
103         DestroyList(L);
104     }
105     InitList(L);
106     FILE *r = fopen(FileName, "r");
107     if (r == NULL) return INFEASIBLE;
108     while(fscanf(r, "%d", &t) != EOF){
109         ListInsert(L, L.length + 1, t);
110     }
111     fclose(r);
112     return OK;
113 }
114
115 status AddList(LISTS &Lists, char ListName[])
116 {
117     InitList(Lists.elem[Lists.length].L);
118     strcpy(Lists.elem[Lists.length].name, ListName);
119     Lists.length++;
```

```
120 return OK;
121 }
122
123 status RemoveList(LISTS &Lists, char ListName[])
124 {
125     for(int i = 0; i < Lists.length; i++)
126     {
127         if(strcmp(ListName, Lists.elem[i].name) == 0)
128         {
129             DestroyList(Lists.elem[i].L);
130             for(int j = i; j < Lists.length - 1; j++)
131             {
132                 Lists.elem[j] = Lists.elem[j + 1];
133             }
134             Lists.length--;
135             return OK;
136         }
137     }
138     return ERROR;
139 }
140
141 int LocateList(LISTS Lists, char ListName[])
142 {
143     for(int i = 0; i < Lists.length; i++)
144     {
145         if(strcmp(ListName, Lists.elem[i].name) == 0)
146         {
147             count = i + 1;
148             // 复制 elem[i].L 到 L
149             if (L.elem != NULL) {
150                 DestroyList(L);
151             }
152             InitList(L);
153             for (int j = 0; j < Lists.elem[i].L.length; j++) {
154                 ElemType e;
155                 GetElem(Lists.elem[i].L, j + 1, e);
156                 ListInsert(L, j + 1, e);
157             }
158             return i + 1;
159         }
160     }
```


华中科技大学课程实验报告

```
161 return 0;
162 }
163
164 // 复制 L 到 elem[i]
165 void copyListToElem(LISTS &Lists, int index, SqList L) {
166     if (Lists.elem[index].L.elem != NULL) {
167         DestroyList(Lists.elem[index].L);
168     }
169     InitList(Lists.elem[index].L);
170     for (int i = 0; i < L.length; i++) {
171         ElemType e;
172         GetElem(L, i + 1, e);
173         ListInsert(Lists.elem[index].L, i + 1, e);
174     }
175 }
176
177 int k = 0;
178 char s1[100];
179
180 /*-----*/
181 int main(){
182     int op = 1, op1 = 1, op0 = 4, result, tp, e;
183     char name[100];
184
185     op0:
186     while(op0)
187     {
188         system("cls");
189         printf("\n\n");
190         printf("\t    Menu for Linear Table On Sequence Structure \n");
191         printf("\t-----\n");
192         printf("\t    1. 多表管理      \t2. 對當前表操作\n");
193         printf("\t    0. 結束\n      ");
194         printf("\t-----\n");
195
196         printf("    請選擇你的操作[0~2]:");
197         scanf("%d", &op0);
198         switch(op0)
199         {
200             case 1:
201                 {
```

```
202         op1 = 100;
203         goto op1;
204         break;
205     }
206     case 2:
207     {
208         op = 100;
209         goto op;
210         break;
211     }
212     case 0:
213     {
214         printf("GG");
215         return 0;
216         break;
217     }
218     default:
219     {
220         printf("無效輸入請重新輸入\n");
221     }
222 }
223 system("PAUSE");
224 }
225 op1:
226 while(op1)
227 {
228     system("cls");
229     printf("\n\n");
230     printf("\t    Menu for Linear Table On Sequence Structure \n");
231     printf("\t-----\n");
232     printf("\t    1. 創立表    \t3. 刪除表\n");
233     printf("\t    2. 查找表    \t0. 返回\n");
234     printf("\t-----\n");
235
236     printf("    請選擇你的操作 [0~3]:\n");
237     scanf("%d", &op1);
238     switch(op1)
239     {
240         case 1:
241         {
242             printf("Name for the new list\n");
```

华中科技大学课程实验报告

```
243         scanf("%s", name);
244         AddList(Lists, name);
245         printf("success\n");
246         break;
247     }
248     case 2:
249     {
250         printf("Name for the list\n");
251         scanf("%s", name);
252         result = LocateList(Lists, name);
253         if (result != 0) printf("success\n");
254         if (result == 0) printf("查無此表\n");
255         break;
256     }
257     case 3:
258     {
259         printf("Delete the List:(input list's name)\n");
260         scanf("%s", name);
261         result = RemoveList(Lists, name);
262         if (result == OK)
263         {
264             printf("OK\n");
265
266         }
267         else
268         {
269             printf("刪除失敗\n");
270             }//if (result == (INFEASIBLE || ERROR)) printf("刪除失敗\n");
271
272         break;
273     }
274     case 0:
275     {
276         op0 = 4;
277         goto op0;
278         break;
279     }
280     default:
281     {
282         printf("輸入有誤，請重新輸入\n");
283         break;
```

华中科技大学课程实验报告

```
284         }
285     }
286     system("PAUSE");
287 }
288 op:
289 while(op){
290     system("cls");
291     printf("\n\n");
292     printf("\t    Menu for Linear Table On Sequence Structure \n");
293     printf("\t-----\n");
294     printf("\t    1. InitList    \t7. LocateElem\n");
295     printf("\t    2. DestroyList \t8. PriorElem\n");
296     printf("\t    3. ClearList   \t9. NextElem \n");
297     printf("\t    4. ListEmpty   \t10. ListInsert\n");
298     printf("\t    5. ListLength  \t11. ListDelete\n");
299     printf("\t    6. GetElem     \t12. ListTraverse\n\n");
300     printf("\t    13. MaxSubArray \t14. SubArrayNum\n");
301     printf("\t    15. sortList   \t16. Savelist\n");
302     printf("\t    17. LoadFile  \t18. WithMultiLists\n");
303     printf("\t    0. turn back\n");
304     printf("\t-----\n");
305     printf("    請選擇你的操作 [0~18]:");
306     scanf("%d", &op);
307     switch(op){
308         case 1:
309             result = InitList(L);
310             if (result == OK) printf("OK\n");
311             if (result == INFEASIBLE) printf("線性表已存在! \n");
312             break;
313         case 2:
314             result = DestroyList(L);
315             if (result == OK) printf("OK\n");
316             if (result == INFEASIBLE) printf("線性表不存在! \n");
317             break;
318         case 3:
319             result = ClearList(L);
320             if (result == OK) printf("OK\n");
321             if (result == INFEASIBLE) printf("線性表不存在! \n");
322             break;
323         case 4:
324             result = ListEmpty(L);
```

```
325     if (result == TRUE) printf("線性表為空! \n");
326     if(result == FALSE) printf("線性表不為空! \n");
327     if (result == INFEASIBLE) printf("線性表不存在! \n");
328     break;
329     case 5:
330     result = ListLength(L);
331     if (result == INFEASIBLE) printf("線性表不存在! \n");
332     else printf("線性表長度為%d\n", result);
333     break;
334     case 6:
335     printf("獲取線性表的第i個元素: ");
336     scanf("%d", &tp);
337     result = GetElem(L, tp, e);
338     if (result == ERROR) printf("查找失敗! \n");
339     if (result == INFEASIBLE) printf("線性表不存在! \n");
340     if(result == OK) printf("第%d個元素為%d\n", tp, e);
341     break;
342     case 7:
343     printf("輸入你要查找的元素: ");
344     scanf("%d", &tp);
345     result = LocateElem(L, tp);
346     if (result == ERROR) printf("查找失敗! \n");
347     else if (result == INFEASIBLE) printf("線性表不存在! \n");
348     else printf("第%d個元素為%d\n", result, tp);
349     break;
350     case 8:
351     printf("輸入你要查找的元素前驅: ");
352     scanf("%d", &tp);
353     result = PriorElem(L, tp, e);
354     if (result == ERROR) printf("查找失敗! \n");
355     else if (result == INFEASIBLE) printf("線性表不存在! \n");
356     else printf("%d的前驅為%d\n", tp, e);
357     break;
358     case 9:
359     printf("輸入你要查找的元素後繼: ");
360     scanf("%d", &tp);
361     result = NextElem(L, tp, e);
362     if (result == ERROR) printf("查找失敗! \n");
363     else if (result == INFEASIBLE) printf("線性表不存在! \n");
364     else printf("%d的後繼為%d\n", tp, e);
365     break;
```

```
366         case 10:
367             printf("輸入在第i個元素前插入的元素: ");
368             scanf("%d%d", &tp, &e);
369             result = ListInsert(L, tp, e);
370             if (result == ERROR) printf("插入失敗! \n");
371             else if (result == INFEASIBLE) printf("線性表不存在! \n");
372             else printf("OK\n");
373             break;
374         case 11:
375             printf("輸入你要刪除的第i個元素: ");
376             scanf("%d", &tp);
377             result = ListDelete(L, tp, e);
378             if (result == ERROR) printf("查找失敗! \n");
379             else if (result == INFEASIBLE) printf("線性表不存在! \n");
380             else printf("已刪除第%d個元素%d\n", tp, e);
381             break;
382         case 12:
383             result = ListTraverse(L);
384             if (result == INFEASIBLE) printf("線性表不存在! \n");
385             break;
386         case 13:
387             result = maxSubArraySum(L);
388             printf("最大子數組和是%d", result);
389             if (result == INFEASIBLE) printf("線性表不存在! \n");
390             break;
391         case 14:
392             printf("設定子數組和");
393             scanf("%d", &k);
394             result = SubArrayNum(L, k);
395             printf("有%d個", result);
396             if (result == INFEASIBLE) printf("線性表不存在! \n");
397             break;
398         case 15:
399             sortList(L);
400             printf("已sort,let try");
401             if (result == INFEASIBLE) printf("線性表不存在! \n");
402             break;
403         case 16:
404             printf("輸入保存文件名");
405             scanf("%s", s1);
406             result = SaveList(L, s1);
```

华中科技大学课程实验报告

```
407         if (result == INFEASIBLE) printf("線性表不存在! \n");
408         break;
409         case 17:
410             printf("輸入加載文件名");
411             scanf("%s", s1);
412             result = LoadList(L, s1);
413             if (result == INFEASIBLE) printf("線性表不存在! \n");
414             break;
415         case 18:
416             result = ListTraverse(L);
417             if (result == INFEASIBLE) printf("線性表不存在! \n");
418             break;
419         case 0:
420             // 操作完把 L 复制回 elem[i]
421             if (count > 0) {
422                 copyListToElem(Lists, count - 1, L);
423             }
424             // 重置 L
425             if (L.elem != NULL) {
426                 DestroyList(L);
427             }
428             InitList(L);
429             op0 = 4;
430             goto op0;
431             break;
432     } //end of switch
433     system("PAUSE");
434 } //end of while
435 return 0;
436 } //end of main()
437
438 status InitList(SqList& L)
439 {
440     if (L.elem == NULL)
441     {
442         L.elem = (ElemType *) malloc(sizeof(ElemType) * LIST_INIT_SIZE);
443         L.length = 0;
444         L.listsize = LIST_INIT_SIZE;
445         return OK;
446     }
447     else
```

```
448 {
449     return INFEASIBLE;
450 }
451 }
452
453 status DestroyList(SqList& L)
454 {
455     if(L.elem != NULL)
456     {
457         free(L.elem);
458         L.elem = NULL;
459         L.length = 0;
460         L.listsize = 0;
461         return OK;
462     }
463     else
464     {
465         return INFEASIBLE;
466     }
467 }
468
469 status ClearList(SqList& L)
470 {
471     if(L.elem != NULL)
472     {
473         L.length = 0;
474         return OK;
475     }
476     else
477     {
478         return INFEASIBLE;
479     }
480 }
481
482 status ListEmpty(SqList L)
483 {
484     if(L.elem == NULL)
485     {
486         return INFEASIBLE;
487     }
488     if(L.length == 0)
```



```
489 {
490     return TRUE;
491 }
492 else
493 {
494     return FALSE;
495 }
496 }
497
498 status ListLength(SqList L)
499 {
500     if(L.elem != NULL)
501     {
502         return L.length;
503     }
504     else
505     {
506         return INFEASIBLE;
507     }
508 }
509
510 status GetElem(SqList L, int i, ElemType &e)
511 {
512     if(i < 1 || i > L.length)
513     {
514         return ERROR;
515     }
516     if(L.elem != NULL)
517     {
518         e = L.elem[i - 1];
519         return OK;
520     }
521     else
522     {
523         return INFEASIBLE;
524     }
525 }
526
527 int LocateElem(SqList L, ElemType e)
528 {
529     if(L.elem != NULL)
```

```
530 {
531     for(int i = 0; i < L.length; i++)
532     {
533         if(e == L.elem[i])
534         {
535             return i + 1;
536         }
537     }
538     return 0;
539 }
540 else
541 {
542     return INFEASIBLE;
543 }
544 }
545
546 status PriorElem(SqList L, ElemType e, ElemType &pre)
547 {
548     if(L.elem != NULL)
549     {
550         for(int i = 1; i < L.length; i++)
551         {
552             if(e == L.elem[i])
553             {
554                 pre = L.elem[i - 1];
555                 return OK;
556             }
557         }
558         return ERROR;
559     }
560 else
561 {
562     return INFEASIBLE;
563 }
564 }
565
566 status NextElem(SqList L, ElemType e, ElemType &next)
567 {
568     if(L.elem != NULL)
569     {
570         for(int i = 1; i < L.length - 1; i++)
```

```
571     {
572         if(e == L.elem[i])
573         {
574             next = L.elem[i + 1];
575             return OK;
576         }
577     }
578     return ERROR;
579 }
580 else
581 {
582     return INFEASIBLE;
583 }
584 }
585
586 status ListInsert(SqList &L, int i, ElemType e)
587 {
588     if(L.elem == NULL) return INFEASIBLE;
589     if(i < 1 || i > L.length + 1) return ERROR;
590     if(L.length >= L.listsize){
591         int *newbase = (int *)realloc(L.elem, (L.listsize + LISTINCREMENT) *
592             sizeof(int));
593         if(newbase == NULL) return 0;
594         L.listsize += LISTINCREMENT;
595         L.elem = newbase;
596     }
597     for(int j = L.length; j > i - 1; j--){
598         L.elem[j] = L.elem[j - 1];
599     }
600     L.elem[i - 1] = e;
601     L.length++;
602     return OK;
603 }
604
605 status ListDelete(SqList &L, int i, ElemType &e)
606 {
607     if(L.elem == NULL)
608     {
609         return INFEASIBLE;
610     }
611     if(i <= 0 || i > L.length)
```

```
611 {
612     return ERROR;
613 }
614 e = L.elem[i - 1];
615 for(int j = i - 1; j < L.length - 1; j++)
616 {
617     L.elem[j] = L.elem[j + 1];
618 }
619 L.length--;
620 return OK;
621 }
622
623 status ListTraverse(SqList L)
624 {
625     if(L.elem == NULL)
626     {
627         return INFEASIBLE;
628     }
629     for(int i = 0; i < L.length; i++)
630     {
631         printf("%d", L.elem[i]);
632         if(i < L.length - 1)
633         {
634             printf(" ");
635         }
636     }
637     printf("\n");
638     return OK;
639 }
```

附录 B 基于链式存储结构线性表实现的源程序

Listing 37 目标程序

```
1  /* Linear Table On Sequence Structure */
2  #include <stdio.h>
3  #include <iostream>
4  #include <algorithm>
5  #include <malloc.h>
6  #include <stdlib.h>
7  #include <string.h>
8  /*-----page 10 on textbook -----*/
9  #define TRUE 1
10 #define FALSE 0
11 #define OK 1
12 #define ERROR 0
13 #define INFEASIBLE -1
14 #define OVERFLOW -2
15
16 typedef int status;
17 typedef int ElemType; //資料元素類型定義
18
19 /*-----page 22 on textbook -----*/
20 #define LIST_INIT_SIZE 100
21 #define LISTINCREMENT 10
22 typedef struct LNode{ //單鏈表（鏈式結構）結點的定義
23     ElemType data;
24     struct LNode *next;
25 }LNode,*LinkList;
26 LinkList L,Ltemp;
27 typedef struct{ //線性表的集合類型定義
28     struct { char name[30];
29             LinkList L;
30     } elem[10];
31     int length;
32 }LISTS;
33 LISTS Lists; //線性表集合的定義Lists
34 /*-----page 19 on textbook -----*/
35 status InitList(LinkList &L);
36 status DestroyList(LinkList &L);
37 status ClearList(LinkList &L);
```

华中科技大学课程实验报告

```
38 status ListEmpty(LinkList L);
39 int ListLength(LinkList L);
40 status GetElem(LinkList L,int i,ElemType &e);
41 status LocateElem(LinkList L,ElemType e);
42 status PriorElem(LinkList L,ElemType e,ElemType &pre);
43 status NextElem(LinkList L,ElemType e,ElemType &next);
44 status ListInsert(LinkList &L,int i,ElemType e);
45 status ListDelete(LinkList &L,int i,ElemType &e);
46 status ListTraverse(LinkList L);
47 status SaveList(LinkList L,char FileName[]);
48 status LoadList(LinkList &L,char FileName[]);
49 status AddList(LISTS &Lists,char ListName[]);
50 status RemoveList(LISTS &Lists,char ListName[]);
51 int LocateList(LISTS Lists,char ListName[]);
52 status reverseList(LinkList L);
53 int RemoveNthFromEnd(LinkList L,int k);
54 int sortList(LinkList L);
55 //简化过
56 /*-----*/
57 int main(){
58     int op=1,result,tp,e,k,op1=1,t;
59     char ch[1000];
60     char name[30];
61     while(op1){
62         system("cls"); printf("\n\n");
63         printf("-----\n");
64         printf("1. 进入多线性表管理    2. 进入单线性表管理\n");
65         printf("0. Exit\n");
66         printf("-----\n");
67         scanf("%d",&op1);
68         switch(op1){
69             case 1:
70                 op=1;
71                 while(op!=0 and op!=4){
72                     t=0;
73                     system("cls"); printf("\n\n");
74                     printf("    多线性表管理!    \n");
75                     printf("-----\n");
76                     printf("        1. AddList    3. LocateList\n");
77                     printf("        2. DestroyList 4. 返回选择页面\n");
78                     printf("        0.Exit\n");
```

```
79     printf("-----\n");
80     printf("  請選擇你的操作[0~4]:");
81     scanf("%d",&op);
82     switch(op){
83     case 1:
84         printf("添加一個線性表，請輸入線性表名稱：");
85         scanf("%s",name);
86         result=AddList(Lists,name);
87         if(result==OK) printf("OK!\n");
88         break;
89     case 2:
90         printf("刪除一個線性表，請輸入線性表名稱：");
91         scanf("%s",name);
92         result=RemoveList(Lists,name);
93         if(result==OK) printf("OK!\n");
94         else printf("線性表不存在！\n");
95         break;
96     case 3:
97         printf("查找一個線性表，請輸入線性表名稱：");
98         scanf("%s",name);
99         result=LocateList(Lists,name);
100        if(result==ERROR) printf("線性表不存在！\n");
101        else{
102            int t;
103            printf("OK! 按1對其進行單線性表操作,按其他按鈕返回");
104            scanf("%d",&t);
105            if(t==1){
106                L=Lists.elem[result-1].L;
107                while(op!=0 and op!=18){
108                    int t=0;
109                    system("cls"); printf("\n\n");
110                    printf("      單線性表管理!      \n");
111                    printf("-----\n");
112                    printf("          1. InitList      7. LocateElem\n");
113                    printf("          2. DestroyList 8. PriorElem\n");
114                    printf("          3. ClearList    9. NextElem \n");
115                    printf("          4. ListEmpty    10. ListInsert\n");
116                    printf("          5. ListLength   11. ListDelete\n");
117                    printf("          6. GetElem      12. ListTraverse\n");
118                    printf("          13. reverseList 14. RemoveNthFromEnd\n");
119                    printf("          15. sortList    16. SaveList\n");
```

```
120         printf("      17. LoadList    18.返回多线性表操作\n");
121         printf("      0. Exit\n");
122         printf("-----\n");
123         printf("  请选择你的操作[0~18]:");
124         scanf("%d",&op);
125         switch(op){
126             case 1:
127                 result=InitList(L);
128                 if (result==OK) printf("OK\n");
129                 if (result==INFEASIBLE) printf("线性表已存在! \n");
130                 break;
131             case 2:
132                 result=DestroyList(L);
133                 if (result==OK) printf("OK\n");
134                 if (result==INFEASIBLE) printf("线性表不存在! \n");
135                 break;
136             case 3:
137                 result=ClearList(L);
138                 if (result==OK) printf("OK\n");
139                 if (result==INFEASIBLE) printf("线性表不存在! \n");
140                 break;
141             case 4:
142                 result=ListEmpty(L);
143                 if (result==TRUE) printf("线性表为空! \n");
144                 if(result==FALSE) printf("线性表不为空! \n");
145                 if (result==INFEASIBLE) printf("线性表不存在! \n");
146                 break;
147             case 5:
148                 result= ListLength(L);
149                 if (result==INFEASIBLE) printf("线性表不存在! \n");
150                 else printf("线性表长度为%d\n",result);
151                 break;
152             case 6:
153                 printf("获取线性表的第i个元素: ");
154                 scanf("%d",&tp);
155                 result=GetElem(L,tp,e);
156                 if (result==ERROR) printf("查找失败! \n");
157                 if (result==INFEASIBLE) printf("线性表不存在! \n");
158                 if(result==OK) printf("第%d个元素为%d\n",tp,e);
159                 break;
160             case 7:
```



```
161         printf("輸入你要查找的元素: ");
162         scanf("%d",&tp);
163         result=LocateElem(L,tp);
164         if (result==ERROR) printf("查找失敗! \n");
165         else if (result==INFEASIBLE)
166             printf("線性表不存在! \n");
167         else printf("第%d個元素為%d\n",result,tp);
168         break;
169     case 8:
170         printf("輸入你要查找的元素前驅: ");
171         scanf("%d",&tp);
172         result=PriorElem(L,tp,e);
173         if (result==ERROR) printf("查找失敗! \n");
174         else if (result==INFEASIBLE)
175             printf("線性表不存在! \n");
176         else printf("%d的前驅為%d\n",tp,e);
177         break;
178     case 9:
179         printf("輸入你要查找的元素後繼: ");
180         scanf("%d",&tp);
181         result=NextElem(L,tp,e);
182         if (result==ERROR) printf("查找失敗! \n");
183         else if (result==INFEASIBLE)
184             printf("線性表不存在! \n");
185         else printf("%d的後繼為%d\n",tp,e);
186         break;
187     case 10:
188         printf("輸入在第i個元素前插入的元素: ");
189         scanf("%d%d",&tp,&e);
190         result=ListInsert(L,tp,e);
191         if (result==ERROR) printf("插入失敗! \n");
192         else if (result==INFEASIBLE)
193             printf("線性表不存在! \n");
194         else printf("OK\n");
195         break;
196     case 11:
197         printf("輸入你要刪除的第i個元素: ");
198         scanf("%d",&tp);
199         result=ListDelete(L,tp,e);
200         if (result==ERROR) printf("查找失敗! \n");
```

```
197         else if (result==INFEASIBLE)
198             printf("線性表不存在! \n");
199         else printf("已刪除第%d個元素%d\n",tp,e);
200         break;
201     case 12:
202         result= ListTraverse(L);
203         if (result==INFEASIBLE) printf("線性表不存在! \n");
204         break;
205     case 13:
206         result=reverseList(L);
207         if (result==INFEASIBLE) printf("線性表不存在! \n");
208         else printf("OK!\n");
209         break;
210     case 14:
211         printf("刪除倒數第n個數:");
212         scanf("%d",&k);
213         result=RemoveNthFromEnd(L,k);
214         if (result==INFEASIBLE)
215             printf("線性表不存在或為空! \n");
216         else if(result == ERROR) printf("刪除失敗! \n");
217         else printf("已刪除倒數第%d個數%d! \n",k,result);
218         break;
219     case 15:
220         result=sortList(L);
221         if (result==INFEASIBLE) printf("線性表不存在! \n");
222         else printf("OK!\n");
223         break;
224     case 16:
225         printf("輸入檔案名: ");
226         scanf("%s",ch);
227         result=SaveList(L,ch);
228         if (result==INFEASIBLE) printf("線性表不存在! \n");
229         else printf("OK\n");
230         break;
231     case 17:
232         printf("輸入檔案名: ");
233         scanf("%s",ch);
234         result=LoadList(L,ch);
235         if (result==INFEASIBLE) printf("線性表已存在! \n");
236         else printf("OK\n");
237         break;
```

```
236         case 18:
237             break;
238         case 0:
239             t=1;
240             break;
241             Lists.elem[result-1].L=L;
242     }//end of switch
243     system("PAUSE");
244 }
245 }
246 }
247         break;
248         case 4:
249             break;
250         case 0:
251             t=1;
252             break;
253     }
254     system("PAUSE");
255 }
256 break;
257
258 case 2:
259     op=1;
260     L=Ltemp;
261     while(op!=0 and op!=18){
262         t=0;
263         system("cls"); printf("\n\n");
264         printf("    單線性表管理!    \n");
265         printf("-----\n");
266         printf("        1. InitList    7. LocateElem\n");
267         printf("        2. DestroyList 8. PriorElem\n");
268         printf("        3. ClearList   9. NextElem \n");
269         printf("        4. ListEmpty   10. ListInsert\n");
270         printf("        5. ListLength  11. ListDelete\n");
271         printf("        6. GetElem     12. ListTraverse\n");
272         printf("        13. reverseList 14. RemoveNthFromEnd\n");
273         printf("        15. sortList   16. SaveList\n");
274         printf("        17. LoadList   18. 返回選擇頁面\n");
275         printf("        0. Exit\n");
276         printf("-----\n");
```

华中科技大学课程实验报告

```
277         printf(" 請選擇你的操作[0~18]:");
278         scanf("%d",&op);
279         switch(op){
280         case 1:
281             result=InitList(L);
282             if (result==OK) printf("OK\n");
283             if (result==INFEASIBLE) printf("線性表已存在! \n");
284             break;
285         case 2:
286             result=DestroyList(L);
287             if (result==OK) printf("OK\n");
288             if (result==INFEASIBLE) printf("線性表不存在! \n");
289             break;
290         case 3:
291             result=ClearList(L);
292             if (result==OK) printf("OK\n");
293             if (result==INFEASIBLE) printf("線性表不存在! \n");
294             break;
295         case 4:
296             result=ListEmpty(L);
297             if (result==TRUE) printf("線性表為空! \n");
298             if(result==FALSE) printf("線性表不為空! \n");
299             if (result==INFEASIBLE) printf("線性表不存在! \n");
300             break;
301         case 5:
302             result= ListLength(L);
303             if (result==INFEASIBLE) printf("線性表不存在! \n");
304             else printf("線性表長度為%d\n",result);
305             break;
306         case 6:
307             printf("獲取線性表的第i個元素: ");
308             scanf("%d",&tp);
309             result=GetElem(L,tp,e);
310             if (result==ERROR) printf("查找失敗! \n");
311             if (result==INFEASIBLE) printf("線性表不存在! \n");
312             if(result==OK) printf("第%d個元素為%d\n",tp,e);
313             break;
314         case 7:
315             printf("輸入你要查找的元素: ");
316             scanf("%d",&tp);
317             result=LocateElem(L,tp);
```

```
318         if (result==ERROR) printf("查找失败! \n");
319     else if (result==INFEASIBLE) printf("线性表不存在! \n");
320     else printf("第%d个元素为%d\n",result,tp);
321     break;
322     case 8:
323     printf("输入你要查找的元素前驱: ");
324     scanf("%d",&tp);
325     result=PriorElem(L,tp,e);
326     if (result==ERROR) printf("查找失败! \n");
327     else if (result==INFEASIBLE) printf("线性表不存在! \n");
328     else printf("%d的前驱为%d\n",tp,e);
329     break;
330     case 9:
331     printf("输入你要查找的元素后继: ");
332     scanf("%d",&tp);
333     result=NextElem(L,tp,e);
334     if (result==ERROR) printf("查找失败! \n");
335     else if (result==INFEASIBLE) printf("线性表不存在! \n");
336     else printf("%d的后继为%d\n",tp,e);
337     break;
338     case 10:
339     printf("输入在第i个元素前插入的元素: ");
340     scanf("%d%d",&tp,&e);
341     result=ListInsert(L,tp,e);
342     if (result==ERROR) printf("插入失败! \n");
343     else if (result==INFEASIBLE) printf("线性表不存在! \n");
344     else printf("OK\n");
345     break;
346     case 11:
347     printf("输入你要删除的第i个元素: ");
348     scanf("%d",&tp);
349     result=ListDelete(L,tp,e);
350     if (result==ERROR) printf("查找失败! \n");
351     else if (result==INFEASIBLE) printf("线性表不存在! \n");
352     else printf("已删除第%d个元素%d\n",tp,e);
353     break;
354     case 12:
355     result= ListTraverse(L);
356     if (result==INFEASIBLE) printf("线性表不存在! \n");
357     break;
358     case 13:
```

```
359         result=reverseList(L);
360         if (result==INFEASIBLE) printf("線性表不存在! \n");
361         else printf("OK!\n");
362         break;
363         case 14:
364             printf("刪除倒數第n個數:");
365             scanf("%d",&k);
366             result=RemoveNthFromEnd(L,k);
367             if (result==INFEASIBLE) printf("線性表不存在或為空! \n");
368             else if(result == ERROR) printf("刪除失敗! \n");
369             else printf("已刪除倒數第%d個數%d! \n",k,result);
370             break;
371         case 15:
372             result=sortList(L);
373             if (result==INFEASIBLE) printf("線性表不存在! \n");
374             else printf("OK!\n");
375             break;
376         case 16:
377             printf("輸入檔案名: ");
378             scanf("%s",ch);
379             result=SaveList(L,ch);
380             if (result==INFEASIBLE) printf("線性表不存在! \n");
381             else printf("OK\n");
382             break;
383         case 17:
384             printf("輸入檔案名: ");
385             scanf("%s",ch);
386             result=LoadList(L,ch);
387             if (result==INFEASIBLE) printf("線性表已存在! \n");
388             else printf("OK\n");
389             break;
390         case 18:
391             Ltemp=L;
392             break;
393         case 0:
394             t=1;
395             break;
396     }//end of switch
397     system("PAUSE");
398 }
399
```

华中科技大学课程实验报告

```
400         break;//case2 break
401         case 0:
402         break;
403
404     }
405     if(t==1) break;
406
407     }//end of while
408     printf("歡迎下次再使用本系統! \n");
409     return 0;
410 }//end of main()
411 /*-----page 23 on textbook -----*/
412 status InitList(LinkList &L)
413 // 線性表L不存在, 構造一個空的線性表, 返回OK, 否則返回INFEASIBLE。
414 {
415     // 請在這裡補充代碼, 完成本關任務
416     /***** Begin *****/
417     if(L != NULL) return INFEASIBLE;
418     L=(LinkList)malloc(sizeof(LNode));
419     L->next=NULL;
420     return OK;
421
422     /***** End *****/
423 }
424 status DestroyList(LinkList &L)
425 // 如果線性表L存在, 銷毀線性表L, 釋放資料元素的空間, 返回OK, 否則返回INFEASIBLE。
426 {
427     // 請在這裡補充代碼, 完成本關任務
428     /***** Begin *****/
429     if(L==NULL){
430         return INFEASIBLE;
431     }
432     LinkList p=L;
433     while (p != NULL) {
434         LinkList temp = p->next;
435         free(p);
436         p = temp;
437     }
438     L = NULL;
439     return OK;
440 }
```

华中科技大学课程实验报告

```
441      /***** End *****/
442  }
443
444  status ClearList(LinkList &L)
445  // 如果线性表L存在，删除线性表L中的所有元素，返回OK，否则返回INFEASIBLE。
446  {
447      // 請在這裡補充代碼，完成本關任務
448      /***** Begin *****/
449      if(L==NULL){
450          return INFEASIBLE;
451      }
452      LinkList p=L->next;
453      while (p != NULL) {
454          LinkList temp = p->next;
455          free(p);
456          p = temp;
457      }
458      L->next = NULL;
459      return OK;
460
461      /***** End *****/
462  }
463
464  status ListEmpty(LinkList L)
465  //
466  // 如果线性表L存在，判断线性表L是否为空，空就返回TRUE，否则返回FALSE；如果线性表L不存在，返回INFEASIBLE。
467  {
468      // 請在這裡補充代碼，完成本關任務
469      /***** Begin *****/
470      if(L==NULL) return INFEASIBLE;
471      if(L->next==NULL) return TRUE;
472      else return FALSE;
473
474      /***** End *****/
475  }
476
477  int ListLength(LinkList L)
478  // 如果线性表L存在，返回线性表L的长度，否则返回INFEASIBLE。
479  {
480      // 請在這裡補充代碼，完成本關任務
481      /***** Begin *****/
```


华中科技大学课程实验报告

```
481     if(L==NULL) return INFEASIBLE;
482     int s=0;
483     LinkList p=L;
484     while(p->next!=NULL){
485         p=p->next;
486         s++;
487     }
488     return s;
489     /***** End *****/
490 }
491 status GetElem(LinkList L,int i,ElemType &e)
492 //
    如果线性表L存在，获取线性表L的第i个元素，保存在e中，返回OK；如果i不合法，返回ERROR；如果线性表L不存在，返回
493 {
494     // 請在這裡補充代碼，完成本關任務
495     /***** Begin *****/
496     if(L==NULL) return INFEASIBLE;
497     int s=0;
498     if(i<1) return ERROR;
499     LinkList p=L;
500     while(p->next!=NULL){
501         s++;
502         p=p->next;
503         if(s==i){
504             e=p->data;
505             return OK;
506         }
507     }
508     return ERROR;
509
510     /***** End *****/
511 }
512
513 status LocateElem(LinkList L,ElemType e)
514 //
    如果线性表L存在，查找元素e在线性表L中的位置序号；如果e不存在，返回ERROR；当线性表L不存在时，返回INFEASIBLE。
515 {
516     // 請在這裡補充代碼，完成本關任務
517     /***** Begin *****/
518     if(L==NULL) return INFEASIBLE;
519     int s=0;
```

华中科技大学课程实验报告

```
520     LinkList p=L;
521     while(p->next!=NULL){
522         s++;
523         p=p->next;
524         if(e==p->data){
525             return s;
526         }
527     }
528     return ERROR;
529
530     /***** End *****/
531 }
532 status PriorElem(LinkList L,ElemType e,ElemType &pre)
533 //
534     如果线性表L存在，获取线性表L中元素e的前驱，保存在pre中，返回OK；如果没有前驱，返回ERROR；如果线性表L不存在，
535 {
536     // 請在這裡補充代碼，完成本關任務
537     /***** Begin *****/
538     if(L==NULL) return INFEASIBLE;
539     int s=0;
540     LinkList p=L,q;
541     while(p->next!=NULL){
542         s++;
543         q=p;
544         p=p->next;
545         if(e==p->data){
546             if(s==1) return ERROR;
547             pre=q->data;
548             return OK;
549         }
550     }
551     return ERROR;
552
553     /***** End *****/
554 }
555 status NextElem(LinkList L,ElemType e,ElemType &next)
556 //
557     如果线性表L存在，获取线性表L元素e的后继，保存在next中，返回OK；如果没有后继，返回ERROR；如果线性表L不存在，
558 {
559     // 請在這裡補充代碼，完成本關任務
560     /***** Begin *****/
```

```
559     if(L==NULL) return INFEASIBLE;
560     LinkList p=L;
561     while(p->next!=NULL){
562         p=p->next;
563         if(e==p->data){
564             if(p->next==NULL) return ERROR;
565             next=p->next->data;
566             return OK;
567         }
568     }
569     return ERROR;
570
571     /***** End *****/
572 }
573 status ListInsert(LinkList &L,int i,ElemType e)
574 //
    如果线性表L存在，将元素e插入到线性表L的第i个元素之前，返回OK；当插入位置不正确时，返回ERROR；如果线性表L不存在
575 {
576     // 請在這裡補充代碼，完成本關任務
577     /***** Begin *****/
578     if(L==NULL) return INFEASIBLE;
579     if(i<1) return ERROR;
580     int s=0;
581     LinkList p=L,q, L1 = (LinkList)malloc(sizeof(LNode));
582     L1->data=e;
583     while(p!=NULL){
584         s++;
585         q=p;
586         p=p->next;
587         if(s==i){
588             L1->next=p;
589             q->next=L1;
590             return OK;
591         }
592     }
593     return ERROR;
594
595     /***** End *****/
596 }
597
598 status ListDelete(LinkList &L,int i,ElemType &e)
```

华中科技大学课程实验报告

599 //

如果线性表L存在，删除线性表L的第i个元素，并保存在e中，返回OK；当删除位置不正确时，返回ERROR；如果线性表L不存在，返回INFEASIBLE。

600 {

601 // 請在這裡補充代碼，完成本關任務

602 ****** Begin ******

603 if(L==NULL) return INFEASIBLE;

604 if(i<1) return ERROR;

605 int s=0;

606 LinkList p=L,q;

607 while(p->next!=NULL){

608 s++;

609 q=p;

610 p=p->next;

611 if(s==i){

612 e=p->data;

613 q->next=p->next;

614 free(p);

615 return OK;

616 }

617 }

618 return ERROR;

619

620 ****** End ******

621 }

622

623 status ListTraverse(LinkList L)

624 //

如果线性表L存在，依次显示线性表中的元素，每个元素间空一格，返回OK；如果线性表L不存在，返回INFEASIBLE。

625 {

626 // 請在這裡補充代碼，完成本關任務

627 ****** Begin ******

628 if(L==NULL) return INFEASIBLE;

629 LinkList p=L;

630 while(p->next!=NULL){

631 p=p->next;

632 printf("%d ",p->data);

633 }

634 return OK;

635

636 ****** End ******

637 }

华中科技大学课程实验报告

```
638 status SaveList(LinkList L, char FileName[])
639 // 如果线性表L存在, 将线性表L的元素写到FileName档中, 返回OK, 否则返回INFEASIBLE。
640 {
641     // 請在這裡補充代碼, 完成本關任務
642     /***** Begin 1 *****/
643     if (L==NULL) {
644         return INFEASIBLE;
645     }
646     FILE *fp = fopen(FileName, "w");
647     LinkList p = L->next;
648     while (p != NULL) {
649         fprintf(fp, "%d\n", p->data);
650         p = p->next;
651     }
652
653     fclose(fp);
654     return OK;
655
656     /***** End 1 *****/
657 }
658
659 status LoadList(LinkList &L, char FileName[])
660 // 如果线性表L不存在, 将FileName档中的资料读到线性表L中, 返回OK, 否则返回INFEASIBLE。
661 {
662     // 請在這裡補充代碼, 完成本關任務
663     /***** Begin 2 *****/
664     if (L != NULL) {
665         return INFEASIBLE;
666     }
667     L = (LinkList)malloc(sizeof(LNode));
668     L->next=NULL;
669     FILE *fp = fopen(FileName, "r");
670     LinkList tail = L;
671     ElemType elem;
672     while (fscanf(fp, "%d", &elem) == 1) { // 逐行讀取數據
673         LinkList newNode = (LinkList)malloc(sizeof(LNode));
674         newNode->data = elem;
675         newNode->next = NULL;
676         tail->next = newNode;
677         tail = newNode;
678     }
```

```
679
680 fclose(fp);
681 return OK;
682
683 /***** End 2 *****/
684 }
685
686 status AddList(LISTS &Lists, char ListName[])
687 // 只需要在Lists中增加一个名称为ListName的空线性表，线性表资料又後臺測試程式插入。
688 {
689 // 請在這裡補充代碼，完成本關任務
690 /***** Begin *****/
691 Lists.elem[Lists.length].L=NULL;
692 InitList(Lists.elem[Lists.length].L);
693 strcpy(Lists.elem[Lists.length].name,ListName);
694 Lists.length++;
695 return OK;
696 /***** End *****/
697 }
698 status RemoveList(LISTS &Lists, char ListName[])
699 // Lists中删除一个名称为ListName的线性表
700 {
701 // 請在這裡補充代碼，完成本關任務
702 /***** Begin *****/
703 for(int i=0;i<Lists.length;i++){
704     if(strcmp(ListName,Lists.elem[i].name)==0){
705         for(int j=i;j<Lists.length-1;j++){
706             Lists.elem[j]=Lists.elem[j+1];
707         }
708         Lists.length--;
709         return OK;
710     }
711 }
712 return ERROR;
713 /***** End *****/
714 }
715 int LocateList(LISTS Lists, char ListName[])
716 // 在Lists中查找一个名称为ListName的线性表，成功返回邏輯序號，否則返回0
717 {
718 // 請在這裡補充代碼，完成本關任務
719 /***** Begin *****/
```

```
720 for(int i=0;i<Lists.length;i++){
721     if(strcmp(Lists.elem[i].name,ListName)==0) return i+1;
722 }
723 return ERROR;
724 /***** End *****/
725 }
726 status reverseList(LinkList L){
727     if(L == NULL) return INFEASIBLE;
728     if (L->next == NULL) return OK;
729     LinkList p=L->next->next,lp=L->next,temp;
730     lp->next = NULL;
731     while( p!=NULL){
732         temp=p->next;
733         p->next=lp;
734         lp=p;
735         p=temp;
736     }
737     L->next=lp;
738     return OK;
739 }
740 int RemoveNthFromEnd(LinkList L,int k){
741     if(L==NULL or L->next == NULL) return INFEASIBLE;
742     if(k<1) return ERROR;
743     LinkList fast=L,slow=L;
744     for(int i=1;i<=k;i++){
745         fast=fast->next;
746         if(fast == NULL) return ERROR;
747     }
748     while(fast->next!=NULL){
749         fast=fast->next;
750         slow=slow->next;
751     }
752     int ans = slow->next->data;
753     slow->next=slow->next->next;
754     return ans;
755 }
756 int sortList(LinkList L){
757     if(L==NULL) return INFEASIBLE;
758     int a[100],len=0;
759     LinkList p=L;
760     while(p->next!=NULL){
```

```
761     p=p->next;
762     a[++len]=p->data;
763 }
764 std::sort(a+1,a+len+1);
765 p=L,len=0;
766 while(p->next!=NULL){
767     p=p->next;
768     p->data=a[++len];
769 }
770 return OK;
771 }
```


附录 C 基于二叉链表二叉树实现的源程序

Listing 38 目标程序

```
1  /* Linear Table On Sequence Structure */
2  #include<bits/stdc++.h>
3  /*-----page 10 on textbook -----*/
4  #define TRUE 1
5  #define FALSE 0
6  #define OK 1
7  #define ERROR 0
8  #define INFEASIBLE -1
9
10 typedef int status;
11 typedef int ElemType; //資料元素類型定義
12
13 /*-----page 22 on textbook -----*/
14 #define LIST_INIT_SIZE 100
15 #define LISTINCREMENT 10
16 typedef int KeyType;
17 typedef struct { //二叉樹結點資料類型定義
18     KeyType key;
19     char others[20];
20 } TElemType;
21 typedef struct BiTNode{ //二叉鏈表結點的定義
22     TElemType data;
23     struct BiTNode *lchild,*rchild;
24 } BiTNode, *BiTree;
25 BiTree T,Ttemp;
26 typedef struct{ //線性表的集合類型定義
27     struct { char name[30];
28             BiTree T;
29     } elem[10];
30     int length;
31 }LISTS;
32 LISTS Lists; //線性表集合的定義Lists
33
34 /*-----page 19 on textbook -----*/
35 status CreateBiTree(BiTree &T,TElemType definition[]);
36 status ClearBiTree(BiTree &T);
37 status DestroyBiTree(BiTree &T);
```

```
38 status BiTreeEmpty(BiTree &T);
39 int BiTreeDepth(BiTree T);
40 BiTNode* LocateNode(BiTree T,KeyType e);
41 status Assign(BiTree &T,KeyType e,TElemType value);
42 BiTNode* GetSibling(BiTree T,KeyType e);
43 status InsertNode(BiTree &T,KeyType e,int LR,TElemType c);
44 status DeleteNode(BiTree &T,KeyType e);
45 status PreOrderTraverse(BiTree T,void (*visit)(BiTree));
46 status InOrderTraverse(BiTree T,void (*visit)(BiTree));
47 status PostOrderTraverse(BiTree T,void (*visit)(BiTree));
48 status LevelOrderTraverse(BiTree T,void (*visit)(BiTree));
49 status SaveBiTree(BiTree T, const char FileName[]);
50 status LoadBiTree(BiTree &T, const char FileName[]);
51 int MaxPathSum(BiTree &T);
52 BiTNode* LowestCommonAncestor(BiTree &T,int e1,int e2);
53 BiTree InvertTree(BiTree &T);
54 status AddList(LISTS &Lists,char ListName[]);
55 status RemoveList(LISTS &Lists,char ListName[]);
56 int LocateList(LISTS Lists,char ListName[]);
57 void visit(BiTree T)
58 {
59     printf(" %d,%s",T->data.key,T->data.others);
60 }
61 int op=1,result,tp,e,k,op1=1,t,lflag,len,ldeep=1,ldeepmax,i_1,LR;
62 BiTNode *p_7,*p_8,*p_16;
63 std::map<int,int> flag;
64 TElemType definition[100],c;
65 char ch[1000];
66 char name[30];
67 //简化过
68 /*-----*/
69 int main() {
70     while (op1) {
71         system("cls");
72         printf("\n\n");
73         printf("-----\n");
74         printf("1.进入多二叉树管理    2.进入单二叉树管理\n");
75         printf("0. Exit\n");
76         printf("-----\n");
77         scanf("%d", &op1);
78         switch (op1) {
```

华中科技大学课程实验报告

```
79         case 1:
80         op = 1;
81         while (op != 0 && op != 4) {
82             t = 0;
83             system("cls");
84             printf("\n\n");
85             printf("    多二叉樹管理!    \n");
86             printf("-----\n");
87             printf("        1. AddTree    3. LocateTree\n");
88             printf("        2. DestroyTree 4. 返回選擇頁面\n");
89             printf("        0.Exit\n");
90             printf("-----\n");
91             printf(" 請選擇你的操作[0~4]:");
92             scanf("%d", &op);
93             switch (op) {
94                 case 1:
95                     printf("添加一個二叉樹，請輸入二叉樹名稱: ");
96                     scanf("%s", name);
97                     result = AddList(Lists, name);
98                     if (result == OK) printf("OK!\n");
99                     break;
100                 case 2:
101                     printf("刪除一個二叉樹，請輸入二叉樹名稱: ");
102                     scanf("%s", name);
103                     result = RemoveList(Lists, name);
104                     if (result == OK) printf("OK!\n");
105                     else printf("二叉樹不存在! \n");
106                     break;
107                 case 3:
108                     printf("查找一個二叉樹，請輸入二叉樹名稱: ");
109                     scanf("%s", name);
110                     result = LocateList(Lists, name);
111                     if (result == ERROR) printf("二叉樹不存在! \n");
112                     else {
113                         int t;
114                         printf("OK!
115                             按1對其進行單二叉樹操作,按其他按鈕返回");
116                         scanf("%d", &t);
117                         if (t == 1) {
118                             T = Lists.elem[result - 1].T;
119                             while (op != 0 && op != 20) {
```

华中科技大学课程实验报告

```
119         int t = 0;
120         system("cls");
121         printf("\n\n");
122         printf("    單二叉樹管理!\n");
123         printf("-----");
124         printf("    1. CreateBiTree\n");
125         printf("    7. Assign\n");
126         printf("    2. DestroyBiTree\n");
127         printf("    8. GetSibling\n");
128         printf("    3. ClearBiTree\n");
129         printf("    9. InsertNode \n");
130         printf("    4. BiTreeEmpty\n");
131         printf("    10. DeleteNode\n");
132         printf("    5. BiTreeDepth\n");
133         printf("    11. PreOrderTraverse\n");
134         printf("    6. LocateNode\n");
135         printf("    12. InOrderTraverse\n");
136         printf("    13. PostOrderTraverse\n");
137         printf("    14. LevelOrderTraverse\n");
138         printf("    15. MaxPathSum\n");
139         printf("    16. LowestCommonAncestor\n");
140         printf("    17. InvertTree\n");
141         printf("    18. SaveBiTree\n");
142         printf("    19. LoadBiTree\n");
143         printf("    20. 返回多線性表操作\n");
144         printf("    0. Exit\n");
145         printf("-----");
146         printf("    請選擇你的操作[0~18]:");
147         scanf("%d", &op);
148         switch (op) {
149             case 1:
150                 if (T != NULL) {
151                     printf("二叉樹已存在!");
152                     break;
153                 }
154                 printf("請輸入定義序列:\n");
155                 i_1 = 0;
```

华中科技大学课程实验报告

```
146         do {
147             scanf("%d%s",
                    &definition[i_1].key,
                    definition[i_1].others);
148         } while
            (definition[i_1++].key
              != -1);
149         lflag = len = 0;
150         flag.clear();
151         result = CreateBiTree(T,
            definition);
152         if (result == OK)
            printf("OK\n");
153         if (result == ERROR)
            printf("關鍵字重複! \n");
154         break;
155         case 2:
156             result = DestroyBiTree(T);
157             if (result == OK)
                printf("OK\n");
158             else
                printf("二叉樹不存在! \n");
159             break;
160         case 3:
161             result = ClearBiTree(T);
162             if (result == OK)
                printf("OK\n");
163             else
                printf("二叉樹不存在! \n");
164             break;
165         case 4:
166             result = BiTreeEmpty(T);
167             if (result == TRUE)
                printf("二叉樹為空! \n");
168             if (result == FALSE)
                printf("二叉樹不為空! \n");
169             if (result == INFEASIBLE)
                printf("二叉樹不存在! \n");
170             break;
171         case 5:
172             result = BiTreeDepth(T);
```

华中科技大学课程实验报告

```
173         ldeep = 1;
174         ldeepmax = 0;
175         if (result == INFEASIBLE)
176             printf("二叉樹不存在! \n");
177         else
178             printf("二叉樹深度為%d\n",
179                 result);
180         break;
181     case 6:
182         printf("輸入你要查找的元素關鍵字: ");
183         scanf("%d", &tp);
184         p_7 = LocateNode(T, tp);
185         if (p_7 == NULL)
186             printf("查找失敗! \n");
187         else printf("%d,%s",
188             p_7->data.key,
189             p_7->data.others);
190         break;
191     case 7:
192         printf("輸入關鍵字與修改為的值: ");
193         scanf("%d%d%s", &tp,
194             &c.key, c.others);
195         result = Assign(T, tp, c);
196         if (result == OK)
197             printf("OK");
198         else
199             printf("修改失敗! \n");
200         break;
201     case 8:
202         printf("輸入你要查找的關鍵字的兄弟: ");
203         scanf("%d", &e);
204         p_8 = GetSibling(T, e);
205         if (p_8 == NULL)
206             printf("沒有兄弟! \n");
207         else printf("%d,%s",
208             p_8->data.key,
209             p_8->data.others);
210         break;
211     case 9:
212         printf("輸入4個元素, 分別表示插入位置, 左右 (0或1)");
```

华中科技大学课程实验报告

```
201         scanf("%d%d%d%s", &e,
202             &LR, &c.key,
203             c.others);
204         result = InsertNode(T, e,
205             LR, c);
206         if (result == OK)
207             printf("OK! \n");
208         else
209             printf("插入失败! \n");
210         break;
211     case 10:
212         printf("输入你要删除的元素关键字: ");
213         scanf("%d", &tp);
214         result = DeleteNode(T,
215             tp);
216         if (result == OK)
217             printf("OK!");
218         else printf("删除失败! ");
219         break;
220     case 11:
221         PreOrderTraverse(T,
222             visit);
223         break;
224     case 12:
225         InOrderTraverse(T, visit);
226         break;
227     case 13:
228         PostOrderTraverse(T,
229             visit);
230         break;
231     case 14:
232         LevelOrderTraverse(T,
233             visit);
234         break;
235     case 15:
236         result = MaxPathSum(T);
237         if (result == -1)
238             printf("二叉树不存在! \n");
239         else
240             printf("最大路径为%d!\n",
241                 result);
```

华中科技大学课程实验报告

```
229         break;
230     case 16:
231         int e1, e2;
232         printf("請輸入兩個關鍵字! : \n");
233         scanf("%d%d", &e1, &e2);
234         p_16 =
                LowestCommonAncestor(T,
                e1, e2);
235         if (p_16 == NULL)
                printf("查找失敗! \n");
236         else printf("%d %s\n",
                p_16->data.key,
                p_16->data.others);
237         break;
238     case 17:
239         T = InvertTree(T);
240         if (T != NULL)
                printf("OK!");
241         else printf("翻轉失敗! ");
242         break;
243     case 18:
244         printf("輸入檔案名: ");
245         scanf("%s", ch);
246         result = SaveBiTree(T,
                ch);
247         if (result == OK)
                printf("OK\n");
248         else
                printf("二叉樹不存在! \n");
249         break;
250     case 19:
251         printf("輸入檔案名: ");
252         scanf("%s", ch);
253         result = LoadBiTree(T,
                ch);
254         if (result == OK)
                printf("OK\n");
255         else
                printf("二叉樹不存在! \n");
256         break;
257     case 20:
```


华中科技大学课程实验报告

```
258                                     break;
259                                     case 0:
260                                         t = 1;
261                                         break;
262                                     Lists.elem[result - 1].T
                                         = T;
263                                     }
264                                     system("PAUSE");
265                                 }
266                            }
267                    }
268                    break;
269                    case 4:
270                        break;
271                    case 0:
272                        t = 1;
273                        break;
274                }
275                system("PAUSE");
276            }
277            break;
278
279            case 2:
280                op = 1;
281                T = Ttemp;
282                while (op != 0 && op != 20) {
283                    int t = 0;
284                    system("cls");
285                    printf("\n\n");
286                    printf("    單二叉樹管理!    \n");
287                    printf("-----\n");
288                    printf("        1. CreateBiTree    7. Assign\n");
289                    printf("        2. DestroyBiTree 8. GetSibling\n");
290                    printf("        3. ClearBiTree    9. InsertNode \n");
291                    printf("        4. BiTreeEmpty    10. DeleteNode\n");
292                    printf("        5. BiTreeDepth    11. PreOrderTraverse\n");
293                    printf("        6. LocateNode     12. InOrderTraverse\n");
294                    printf("        13. PostOrderTraverse 14.
                        LevelOrderTraverse\n");
295                    printf("        15. MaxPathSum    16.
                        LowestCommonAncestor\n");
```

```
296         printf("      17. InvertTree   18. SaveBiTree\n");
297         printf("      19. LoadBiTree  20.返回選擇頁面\n");
298         printf("      0. Exit\n");
299         printf("-----\n");
300         printf(" 請選擇你的操作[0~18]:");
301         scanf("%d", &op);
302         switch (op) {
303             case 1:
304                 if (T != NULL) {
305                     printf("二叉樹已存在！");
306                     break;
307                 }
308                 printf("請輸入定義序列:\n");
309                 i_1 = 0;
310                 do {
311                     scanf("%d%s", &definition[i_1].key,
312                             definition[i_1].others);
313                 } while (definition[i_1++].key != -1);
314                 lflag = len = 0;
315                 flag.clear();
316                 result = CreateBiTree(T, definition);
317                 if (result == OK) printf("OK\n");
318                 if (result == ERROR) printf("關鍵字重複！\n");
319                 break;
320             case 2:
321                 result = DestroyBiTree(T);
322                 if (result == OK) printf("OK\n");
323                 else printf("二叉樹不存在！\n");
324                 break;
325             case 3:
326                 result = ClearBiTree(T);
327                 if (result == OK) printf("OK\n");
328                 else printf("二叉樹不存在！\n");
329                 break;
330             case 4:
331                 result = BiTreeEmpty(T);
332                 if (result == TRUE) printf("二叉樹為空！\n");
333                 if (result == FALSE) printf("二叉樹不為空！\n");
334                 if (result == INFEASIBLE) printf("二叉樹不存在！\n");
335                 break;
336             case 5:
```

```
336         result = BiTreeDepth(T);
337         ldeep = 1;
338         ldeepmax = 0;
339         if (result == INFEASIBLE) printf("二叉樹不存在! \n");
340         else printf("二叉樹深度為%d\n", result);
341         break;
342     case 6:
343         printf("輸入你要查找的元素關鍵字: ");
344         scanf("%d", &tp);
345         p_7 = LocateNode(T, tp);
346         if (p_7 == NULL) printf("查找失敗! \n");
347         else printf("%d,%s", p_7->data.key,
348                     p_7->data.others);
349         break;
350     case 7:
351         printf("輸入關鍵字與修改為的值: ");
352         scanf("%d%d%s", &tp, &c.key, c.others);
353         result = Assign(T, tp, c);
354         if (result == OK) printf("OK");
355         else printf("修改失敗! \n");
356         break;
357     case 8:
358         printf("輸入你要查找的關鍵字的兄弟: ");
359         scanf("%d", &e);
360         p_8 = GetSibling(T, e);
361         if (p_8 == NULL) printf("沒有兄弟! \n");
362         else printf("%d,%s", p_8->data.key,
363                     p_8->data.others);
364         break;
365     case 9:
366         printf("輸入4個元素, 分別表示插入位置, 左右(0或1, -1表示作為根節點), 插入位置");
367         scanf("%d%d%d%s", &e, &LR, &c.key, c.others);
368         result = InsertNode(T, e, LR, c);
369         if (result == OK) printf("OK! \n");
370         else printf("插入失敗! \n");
371         break;
372     case 10:
373         printf("輸入你要刪除的元素關鍵字: ");
374         scanf("%d", &tp);
375         result = DeleteNode(T, tp);
376         if (result == OK) printf("OK!");
```

```
375         else printf("刪除失敗! ");
376         break;
377         case 11:
378             PreOrderTraverse(T, visit);
379             break;
380         case 12:
381             InOrderTraverse(T, visit);
382             break;
383         case 13:
384             PostOrderTraverse(T, visit);
385             break;
386         case 14:
387             LevelOrderTraverse(T, visit);
388             break;
389         case 15:
390             result = MaxPathSum(T);
391             if (result == -1) printf("二叉樹不存在! \n");
392             else printf("最大路徑為%d!\n", result);
393             break;
394         case 16:
395             int e1, e2;
396             printf("請輸入兩個關鍵字! : \n");
397             scanf("%d%d", &e1, &e2);
398             p_16 = LowestCommonAncestor(T, e1, e2);
399             if (p_16 == NULL) printf("查找失敗! \n");
400             else printf("%d %s\n", p_16->data.key,
401                          p_16->data.others);
402             break;
403         case 17:
404             T = InvertTree(T);
405             if (T != NULL) printf("OK!");
406             else printf("翻轉失敗! ");
407             break;
408         case 18:
409             printf("輸入檔案名: ");
410             scanf("%s", ch);
411             result = SaveBiTree(T, ch);
412             if (result == OK) printf("OK\n");
413             else printf("二叉樹不存在! \n");
414             break;
415         case 19:
```

华中科技大学课程实验报告

```
415         printf("輸入檔案名: ");
416         scanf("%s", ch);
417         result = LoadBiTree(T, ch);
418         if (result == OK) printf("OK\n");
419         else printf("二叉樹不存在! \n");
420         break;
421         case 20:
422             Ttemp = T;
423             break;
424         case 0:
425             t = 1;
426             break;
427     }
428     system("PAUSE");
429 }
430 break;
431 case 0:
432 break;
433
434 }
435 if (t == 1) break;
436
437 }
438 printf("歡迎下次再使用本系統! \n");
439 return 0;
440 }//end of main()
441 /*-----page 23 on textbook -----*/
442 status CreateBiTree(BiTree &T,TElemType definition[])
443 /*根據帶空枝的二叉樹先根遍歷序列definition構造一棵二叉樹，將根節點指針賦值給T並返回OK，
444 如果有相同的關鍵字，返回ERROR。此題允許通過增加其它函數輔助實現本關任務*/
445 {
446     // 請在這裡補充代碼，完成本關任務
447     /***** Begin *****/
448     if(definition[len].key == -1){
449         T=NULL;
450         return OK;
451     }
452     if(definition[len].key == 0){
453         T=NULL;
454         len++;
455         return OK;
```

```
456     }
457     T=(BiTree)malloc(sizeof(BiTNode));
458     if(flag[definition[len].key] == 1){
459         lflag=1;
460     }
461     flag[definition[len].key] = 1;
462     T->data.key = definition[len].key;
463     strcpy(T->data.others , definition[len].others);
464     T->lchild=T->rchild = NULL;
465     len++;
466     CreateBiTree(T->lchild,definition);
467     CreateBiTree(T->rchild,definition);
468     if(lflag == 1 ) return ERROR;
469     return OK;
470     /***** End *****/
471 }
472
473 status DestroyBiTree(BiTree &T)
474 //將二叉樹設置成空，並刪除所有結點，釋放結點空間
475 {
476     // 請在這裡補充代碼，完成本關任務
477     /***** Begin *****/
478     if(T==NULL) return INFEASIBLE;
479     if(T->lchild!=NULL) ClearBiTree(T->lchild);
480     if(T->rchild!=NULL) ClearBiTree(T->rchild);
481     free(T);
482     T = NULL ;
483     return OK;
484     /***** End *****/
485 }
486
487 status ClearBiTree(BiTree &T)
488 //將二叉樹設置成空，並刪除所有結點，釋放結點空間
489 {
490     // 請在這裡補充代碼，完成本關任務
491     /***** Begin *****/
492     if(T==NULL) return INFEASIBLE;
493     T->lchild = NULL;
494     T->rchild = NULL;
495     T->data.key = 0;
496     strcpy(T->data.others , "");
```

华中科技大学课程实验报告

```
497     return OK;
498     /***** End *****/
499 }
500
501 status BiTreeEmpty(BiTree &T)
502 //
    如果线性表L存在，判断线性表L是否为空，空就返回TRUE，否则返回FALSE；如果线性表L不存在，返回INFEASIBLE。
503 {
504     // 請在這裡補充代碼，完成本關任務
505     /***** Begin *****/
506     if(T==NULL) return INFEASIBLE;
507     if(T->data.key==0) return TRUE;
508     else return FALSE;
509
510     /***** End *****/
511 }
512
513 int BiTreeDepth(BiTree T)
514 //求二叉樹T的深度
515 {
516     // 請在這裡補充代碼，完成本關任務
517     /***** Begin *****/
518     if(T==NULL) return 0;
519     if(T->lchild!=NULL){
520         ldeep++;
521         BiTreeDepth(T->lchild);
522         ldeep--;
523     }
524     if(T->rchild!=NULL){
525         ldeep++;
526         BiTreeDepth(T->rchild);
527         ldeep--;
528     }
529     ldeepmax=std::max(ldeepmax,ldeep);
530     return ldeepmax;
531     /***** End *****/
532 }
533
534 BiTNode* LocateNode(BiTree T,KeyType e)
535 //查找結點
536 {
```

华中科技大学课程实验报告

```
537 // 請在這裡補充代碼，完成本關任務
538 /***** Begin *****/
539 if(T==NULL) return NULL;
540 if(T->data.key == e) return T;
541 BiTNode* k=LocateNode(T->lchild,e);
542 if(k == NULL) return LocateNode(T->rchild,e);
543 else return k;
544
545 /***** End *****/
546 }
547
548 bool check_Assign(BiTree T,int k,BiTNode* now){
549     if(T==NULL) return true;
550     if(T!=now && T->data.key==k) return false;
551     return check_Assign(T->lchild,k,now)&&check_Assign(T->rchild,k,now);
552 }
553 status Assign(BiTree &T,KeyType e,TElemType value)
554 //實現結點賦值。此題允許通過增加其它函數輔助實現本關任務
555 {
556     // 請在這裡補充代碼，完成本關任務
557     /***** Begin *****/
558     BiTNode* p= LocateNode(T,e);
559     if(p==NULL) return ERROR;
560     if(value.key == e ){
561         p->data = value;
562         return OK;
563     }
564     else {
565         if(check_Assign(T,value.key,p)){
566             p->data = value;
567             return OK;
568         }
569         else return ERROR;
570     }
571
572     /***** End *****/
573 }
574
575 BiTNode* GetSibling(BiTree T,KeyType e)
576 //實現獲得兄弟結點
577 {
```


华中科技大学课程实验报告

```
578 // 請在這裡補充代碼，完成本關任務
579 /***** Begin *****/
580 if (T == NULL) {
581     return NULL;
582 }
583
584 // 檢查左孩子是否是目標節點
585 if (T->lchild != NULL && T->lchild->data.key == e) {
586     return T->rchild;
587 }
588
589 // 檢查右孩子是否是目標節點
590 if (T->rchild != NULL && T->rchild->data.key == e) {
591     return T->lchild;
592 }
593
594 // 遞迴在左子樹中查找
595 BiTNode* leftResult = GetSibling(T->lchild, e);
596 if (leftResult != NULL) {
597     return leftResult;
598 }
599
600 // 遞迴在右子樹中查找
601 return GetSibling(T->rchild, e);
602
603 /***** End *****/
604 }
605
606 bool check_Insert(BiTree T,int k){
607     if(T==NULL) return true;
608     if(T->data.key==k) return false;
609     return check_Insert(T->lchild,k)&&check_Insert(T->rchild,k);
610 }
611 status InsertNode(BiTree &T,KeyType e,int LR,TElemType c)
612 //插入結點。此題允許通過增加其它函數輔助實現本關任務
613 {
614     // 請在這裡補充代碼，完成本關任務
615     /***** Begin *****/
616     BiTNode* temp=(BiTree)malloc(sizeof(BiTNode));
617     temp->data = c;
618     temp->lchild=temp->rchild=NULL;
```

```
619     if(LR == -1){
620         temp->rchild = T;
621         T = temp;
622         return OK;
623     }
624     BiTNode* p=LocateNode(T,e);
625     if(p==NULL) return ERROR;
626     if(!check_Insert(T,c.key)) return ERROR;
627     if(LR==0){
628         temp->rchild=p->lchild;
629         p->lchild=temp;
630     }
631     if(LR==1){
632         temp->rchild=p->rchild;
633         p->rchild=temp;
634     }
635     return OK;
636     /***** End *****/
637 }
638
639 status DeleteNode(BiTree &T,KeyType e)
640 //删除结点。此题允许通过增加其它函数辅助实现本关任务
641 {
642     // 請在這裡補充代碼，完成本關任務
643     /***** Begin *****/
644     if(T==NULL) return ERROR;
645     if(T->data.key == e ){
646         if(T->lchild==NULL && T->rchild == NULL){
647             free(T);
648             T=NULL;
649         }
650         else if(T->lchild != NULL && T->rchild!=NULL) {
651             BiTree p = T->lchild;
652             while(p->rchild!=NULL) p = p->rchild;
653             p->rchild =T->rchild;
654             BiTree temp = T;
655             T= T->lchild;
656             free(temp);
657         }
658         else if( T->lchild != NULL){
659             BiTree temp = T;
```

华中科技大学课程实验报告

```
660         T= T->lchild;
661         free(temp);
662     }
663     else if( T->rchild != NULL){
664         BiTree temp = T;
665         T= T->rchild;
666         free(temp);
667     }
668     return OK;
669 }
670 if (DeleteNode(T->lchild,e) == OK) return OK;
671 if (DeleteNode(T->rchild,e) == OK) return OK;
672 return ERROR;
673
674 /***** End *****/
675 }
676
677 status PreOrderTraverse(BiTree T,void (*visit)(BiTree))
678 //先序遍历二叉树T
679 {
680     // 請在這裡補充代碼，完成本關任務
681     /***** Begin *****/
682     printf(" %d,%s",T->data.key,T->data.others);
683     if(T->lchild!=NULL) PreOrderTraverse(T->lchild,visit);
684     if(T->rchild!=NULL) PreOrderTraverse(T->rchild,visit);
685     /***** End *****/
686 }
687
688 status InOrderTraverse(BiTree T,void (*visit)(BiTree))
689 //中序遍历二叉树T
690 {
691     // 請在這裡補充代碼，完成本關任務
692     /***** Begin *****/
693     if(T->lchild!=NULL) InOrderTraverse(T->lchild,visit);
694     printf(" %d,%s",T->data.key,T->data.others);
695     if(T->rchild!=NULL) InOrderTraverse(T->rchild,visit);
696
697     /***** End *****/
698 }
699
700 status PostOrderTraverse(BiTree T,void (*visit)(BiTree))
```

华中科技大学课程实验报告

```
701 //後序遍歷二叉樹T
702 {
703     // 請在這裡補充代碼，完成本關任務
704     /***** Begin *****/
705     if(T->lchild!=NULL) PostOrderTraverse(T->lchild,visit);
706     if(T->rchild!=NULL) PostOrderTraverse(T->rchild,visit);
707     printf(" %d,%s",T->data.key,T->data.others);
708     /***** End *****/
709 }
710 status LevelOrderTraverse(BiTree T,void (*visit)(BiTree))
711 //按層遍歷二叉樹T
712 {
713     // 請在這裡補充代碼，完成本關任務
714     /***** Begin *****/
715     std::queue<BiTree> q;
716     BiTree p;
717     q.push(T);
718     while(!q.empty()){
719         p = q.front();
720         q.pop();
721         printf(" %d,%s",p->data.key,p->data.others);
722         if(p->lchild!=NULL) q.push(p->lchild);
723         if(p->rchild!=NULL) q.push(p->rchild);
724     }
725
726     /***** End *****/
727 }
728
729 int MaxPathSum(BiTree &T){
730     if (T == NULL) {
731         return -1;
732     }
733     int leftMax = MaxPathSum(T->lchild);
734     int rightMax = MaxPathSum(T->rchild);
735     int maxPath = (leftMax > rightMax ? leftMax : rightMax) + 1;
736     return maxPath;
737 }
738
739 BiTNode* LowestCommonAncestor(BiTree &T,int e1,int e2){
740     if (T == NULL) return NULL;
741     if (T->data.key == e1 || T->data.key == e2)
```

```
742     return T;
743     BiTree left = LowestCommonAncestor(T->lchild, e1, e2);
744     BiTree right = LowestCommonAncestor(T->rchild, e1, e2);
745     if (left && right)
746         return T;
747     return left ? left : right;
748 }
749
750 BiTree InvertTree(BiTree &T) {
751     if (T == NULL) {
752         return NULL; // 空樹直接返回
753     }
754
755     // 交換當前節點的左右子樹
756     BiTree temp = T->lchild;
757     T->lchild = T->rchild;
758     T->rchild = temp;
759
760     // 遞迴翻轉新的左右子樹（原右、左子樹）
761     InvertTree(T->lchild);
762     InvertTree(T->rchild);
763
764     return T; // 返回翻轉後的根節
765 }
766
767 status SaveBiTree(BiTree T, const char FileName[])
768 //將二叉樹的結點數據寫入到文件FileName中
769 {
770     // 請在這裡補充代碼，完成本關任務
771     /***** Begin 1 *****/
772     int s = 1, flag=1, lq;
773     std::queue<BiTree> q;
774     BiTree p;
775     FILE *fp = fopen(FileName, "w");
776     q.push(T);
777     while(flag==1){
778         flag=0;
779         lq=q.size();
780         for(int i=1; i<=lq; i++){
781             p = q.front();
782             q.pop();
```

华中科技大学课程实验报告

```
783         if(p==NULL){
784             q.push(NULL);
785             q.push(NULL);
786             s++;
787             continue;
788         }
789         fprintf(fp, "%d %d %s\n", s ,p->data.key,p->data.others);
790         if(p->lchild!=NULL or p->rchild!=NULL) flag = 1;
791         q.push(p->lchild);
792         q.push(p->rchild);
793         s++;
794     }
795 }
796 fprintf(fp, "%d %d %s\n", 0 ,0,NULL);
797 fclose(fp);
798 return OK;
799
800 /***** End 1 *****/
801 }
802 status LoadBiTree(BiTree &T, const char FileName[])
803 //讀入文件FileName的結點數據，創建二叉樹
804 {
805     // 請在這裡補充代碼，完成本關任務
806     /***** Begin 2 *****/
807     FILE *fp = fopen(FileName, "r");
808     TElemType d[100];
809     int ans,i=0,kt=1;
810     BiTNode *p[100]={NULL};
811     while (1){
812         fscanf(fp, "%d%d%s",&kt,&d[i].key,d[i].others);
813         if(kt==0) break;
814         p[kt]=(BiTNode *)malloc(sizeof(BiTNode));
815         p[kt]->data=d[i];
816         p[kt]->lchild=NULL;
817         p[kt]->rchild=NULL;
818         if (kt!=1){
819             if (kt%2==0) p[kt/2]->lchild=p[kt];
820             else p[kt/2]->rchild=p[kt];
821         }
822         i++;
823     }
```

```
824     T=p[1];
825     fclose(fp);
826     return OK;
827     /***** End 2 *****/
828 }
829
830 status AddList(LISTS &Lists,char ListName[])
831 // 只需要在Lists中增加一个名称为ListName的空线性表，线性表资料又後臺測試程式插入。
832 {
833     // 請在這裡補充代碼，完成本關任務
834     /***** Begin *****/
835     Lists.elem[Lists.length].T=(BiTree)malloc(sizeof(BiTNode));
836     Lists.elem[Lists.length].T=NULL;
837     strcpy(Lists.elem[Lists.length].name,ListName);
838     Lists.length++;
839     return OK;
840     /***** End *****/
841 }
842 status RemoveList(LISTS &Lists,char ListName[])
843 // Lists中删除一个名称为ListName的线性表
844 {
845     // 請在這裡補充代碼，完成本關任務
846     /***** Begin *****/
847     for(int i=0;i<Lists.length;i++){
848         if(strcmp(ListName,Lists.elem[i].name)==0){
849             for(int j=i;j<Lists.length-1;j++){
850                 Lists.elem[j]=Lists.elem[j+1];
851             }
852             Lists.length--;
853             return OK;
854         }
855     }
856     return ERROR;
857     /***** End *****/
858 }
859 int LocateList(LISTS Lists,char ListName[])
860 // 在Lists中查找一个名称为ListName的线性表，成功返回邏輯序號，否則返回0
861 {
862     // 請在這裡補充代碼，完成本關任務
863     /***** Begin *****/
864     for(int i=0;i<Lists.length;i++){
```

```
865         if(strcmp(Lists.elem[i].name,ListName)==0) return i+1;
866     }
867     return ERROR;
868     /***** End *****/
869 }
```


附录 D 基于邻接表图实现的源程序

Listing 39 目标程序

```
1  /* Linear Table On Sequence Structure */
2  #include<bits/stdc++.h>
3  /*-----page 10 on textbook -----*/
4  #define TRUE 1
5  #define FALSE 0
6  #define OK 1
7  #define ERROR 0
8  #define INFEASIBLE -1
9  #define MAX_VERTEX_NUM 20
10
11 typedef int status;
12 typedef int ElemType; //資料元素類型定義
13
14 /*-----page 22 on textbook -----*/
15 #define LIST_INIT_SIZE 100
16 #define LISTINCREMENT 10
17 typedef int KeyType;
18 typedef enum {DG,DN,UDG,UDN} GraphKind;
19 typedef struct {
20     KeyType key;
21     char others[20];
22 } VertexType; //頂點類型定義
23
24 typedef struct ArcNode { //表結點類型定義
25     int adjvex; //頂點位置編號
26     struct ArcNode *nextarc; //下一個表結點指標
27 } ArcNode;
28 typedef struct VNode{ //頭結點及其陣列類型定義
29     VertexType data; //頂點信息
30     ArcNode *firstarc; //指向第一條弧
31 } VNode,AdjList[MAX_VERTEX_NUM];
32 typedef struct { //鄰接表的類型定義
33     AdjList vertices; //頭結點陣列
34     int vexnum,arcnum; //頂點數、弧數
35     GraphKind kind; //圖的類型
36 } ALGraph;
37 typedef struct{ //線性表的集合類型定義
```

```
38     struct { char name[30];
39             ALGraph G;
40     } elem[10];
41     int length;
42 }LISTS;
43 LISTS Lists;    //线性表集合的定义Lists
44
45 /*-----page 19 on textbook -----*/
46 status CreateCraph(ALGraph &G,VertexType V[],KeyType VR[][2]);
47 status DestroyGraph(ALGraph &G);
48 status LocateVex(ALGraph G,KeyType u);
49 status PutVex(ALGraph &G,KeyType u,VertexType value);
50 int FirstAdjVex(ALGraph G,KeyType u);
51 int NextAdjVex(ALGraph G,KeyType v,KeyType w);
52 status InsertVex(ALGraph &G,VertexType v);
53 status DeleteVex(ALGraph &G,KeyType v);
54 status InsertArc(ALGraph &G,KeyType v,KeyType w);
55 status DeleteArc(ALGraph &G,KeyType v,KeyType w);
56 status DFSTraverse(ALGraph &G,void (*visit)(VertexType));
57 status BFSTraverse(ALGraph &G,void (*visit)(VertexType));
58 int VerticesSetLessThanK(ALGraph &G,int v,int k);
59 int ShortestPathLength(ALGraph &G,int v,int k);
60 int ConnectedComponentsNums(ALGraph &G);
61 status SaveGraph(ALGraph G, char FileName[]);
62 status LoadGraph(ALGraph &G, char FileName[]);
63
64 status AddList(LISTS &Lists,char ListName[]);
65 status RemoveList(LISTS &Lists,char ListName[]);
66 int LocateList(LISTS Lists,char ListName[]);
67 void visit(VertexType v)
68 {
69     printf(" %d %s",v.key,v.others);
70 }
71 int op=1,result,tp,e,k,op1=1,t,len,i_1;
72 std::map<int,int> flagdfs,flagbfs,flagsave;
73 std::queue<int> q;
74 VertexType V[30],c;
75 KeyType VR[100][2];
76 char ch[1000];
77 char name[30];
78 ALGraph G,Gtemp;
```

华中科技大学课程实验报告

```
79 //简化过
80 /*-----*/
81 int main(){
82
83     while(op1){
84         system("cls"); printf("\n\n");
85         printf("-----\n");
86         printf("1.进入多图管理    2.进入单图管理\n");
87         printf("0. Exit\n");
88         printf("-----\n");
89         scanf("%d",&op1);
90         switch(op1){
91             case 1:
92                 op=1;
93                 while(op!=0 and op!=4){
94                     t=0;
95                     system("cls"); printf("\n\n");
96                     printf("    多图管理!    \n");
97                     printf("-----\n");
98                     printf("        1. AddGraph    3. LocateGraph\n");
99                     printf("        2. DestroyGraph 4. 返回选择页面\n");
100                    printf("        0.Exit\n");
101                    printf("-----\n");
102                    printf("    请选择你的操作[0~4]:");
103                    scanf("%d",&op);
104                    switch(op){
105                        case 1:
106                            printf("添加一个图，请输入图名称：");
107
108                            scanf("%s",name);
109                            result=AddList(Lists,name);
110                            if(result==OK) printf("OK!\n");
111                            break;
112                        case 2:
113                            printf("删除一个图，请输入图名称：");
114                            scanf("%s",name);
115                            result=RemoveList(Lists,name);
116                            if(result==OK) printf("OK!\n");
117                            else printf("图不存在! \n");
118                            break;
119                        case 3:
```

```
120         printf("查找一個圖，請輸入圖名稱：");
121         scanf("%s",name);
122         result=LocateList(Lists,name);
123         if(result==ERROR) printf("圖不存在！\n");
124         else{
125             int t;
126             printf("OK! 按1對其進行單圖操作,按其他按鍵返回");
127             scanf("%d",&t);
128             if(t==1){
129                 G=Lists.elem[result-1].G;
130                 while(op!=0 and op!=18){
131                     int t=0;
132                     system("cls"); printf("\n\n");
133                     printf("      單圖管理!      \n");
134                     printf("-----\n");
135                     printf("          1. CreateGraph    7. InsertVex\n");
136                     printf("          2. DestroyGraph 8. DeleteVex \n");
137                     printf("          3. LocateVex     9. InsertArc \n");
138                     printf("          4. PutVex    10. DeleteArc\n");
139                     printf("          5. FirstAdjVex 11. DFSTraverse\n");
140                     printf("          6. NextAdjVex  12. BFSTraverse\n");
141                     printf("          13. VerticesSetLessThanK 14.
142                        ShortestPathLength\n");
143                     printf("          15. ConnectedComponentsNums 16.
144                        SaveGraph\n");
145                     printf("          17. LoadGraph
146                        18.返回多線性表操作\n");
147                     printf("          0. Exit\n");
148                     printf("-----\n");
149                     printf("      請選擇你的操作[0~18]:");
150                     scanf("%d",&op);
151                     switch(op){
152                         case 1:
153                             if(G.vexnum!=0){
154                                 printf("圖已存在！");
155                                 break;
156                             }
157                             printf("請輸入定義序列:\n");
158                             i_1=0;
159                             do {
160                                 scanf("%d%s",&V[i_1].key,V[i_1].others);
```

```
158         } while(V[i_1++].key!=-1);
159         i_1=0;
160         do {
161             scanf("%d%d",&VR[i_1][0],&VR[i_1][1]);
162         } while(VR[i_1++][0]!=-1);
163         result=CreateCraph(G,V,VR);
164         if (result==OK) printf("OK\n");
165         if (result==ERROR) printf("輸入錯誤! \n");
166         break;
167         case 2:
168             result=DestroyGraph(G);
169             if (result==OK) printf("OK\n");
170             else printf("圖不存在! \n");
171             break;
172         case 3:
173             printf("輸入查找的關鍵字: ");
174             scanf("%d",&tp);
175             result=LocateVex(G,tp);
176             if (result!=-1) printf("%d
177                 %s\n",G.vertices[result].data.key,G.vertices[result].data.others);
178             else printf("ERROR! \n");
179             break;
180             case 4:
181                 printf("輸入關鍵字與修改為的值: ");
182                 scanf("%d%d%s",&tp,&c.key,c.others);
183                 result=PutVex(G,tp,c);
184                 if (result==OK) printf("OK! \n");
185                 else printf("ERROR! \n");
186                 break;
187                 case 5:
188                     printf("輸入要查找的關鍵字: ");
189                     scanf("%d",&tp);
190                     result= FirstAdjVex(G,tp);
191                     if (result==0) printf("ERROR! \n");
192                     else printf("%d
193                         %s\n",G.vertices[result].data.key,G.vertices[result].data.others);
194                     break;
195                     case 6:
196                         printf("輸入v和w: ");
197                         scanf("%d%d",&tp,&e);
198                         result= NextAdjVex(G,tp,e);
```

```
197         if (result== -1) printf("ERROR! \n");
198     else printf("%d
        %s\n",G.vertices[result].data.key,G.vertices[result].data.others);
199     break;
200     case 7:
201     printf("輸入插入的點: ");
202     scanf("%d%s",&c.key,c.others);
203     result=InsertVex(G,c);
204     if(result == OK) printf("OK! \n");
205     else printf("ERROR! \n");
206     break;
207     case 8:
208     printf("輸入你要刪除的關鍵字: ");
209     scanf("%d",&e);
210     result=DeleteVex(G,e);
211     if (result==OK) printf("OK! \n");
212     else printf("ERROR! \n");
213     break;
214     case 9:
215     printf("輸入插入的弧<v,w>");
216     scanf("%d%d",&e,&tp);
217     result=InsertArc(G,e,tp);
218     if (result==OK) printf("OK! \n");
219     else printf("插入失敗! \n");
220     break;
221     case 10:
222     printf("輸入插入的弧<v,w>");
223     scanf("%d%d",&e,&tp);
224     result=DeleteArc(G,e,tp);
225     if (result==OK) printf("OK! \n");
226     else printf("插入失敗! \n");
227     break;
228     case 11:
229     flagdfs.clear();
230     DFSTraverse(G,visit);
231     break;
232     case 12:
233     flagbfs.clear();
234     while (!q.empty()) {
235     q.pop();
236     }
```

```
237         BFSTraverse(G,visit);
238         break;
239     case 13:
240         flagbfs.clear();
241         while (!q.empty()) {
242             q.pop();
243         }
244         printf("輸入v與k: ");
245         scanf("%d%d",&tp,&e);
246         result=VerticesSetLessThanK(G,tp,e);
247         if (result==ERROR) printf("頂點不存在! \n");
248         break;
249     case 14:
250
251         flagbfs.clear();
252         while (!q.empty()) {
253             q.pop();
254         }
255         printf("輸入v與w: ");
256         scanf("%d%d",&tp,&e);
257         result=ShortestPathLength(G,tp,e);
258         if (result== -1) printf("不存在! \n");
259         else printf("距離為%d! \n",result);
260         break;
261     case 15:
262         flagbfs.clear();
263         while (!q.empty()) {
264             q.pop();
265         }
266         result=ConnectedComponentsNums(G);
267         if (result== -1) printf("圖不存在! \n");
268         else printf("連通分量有%d個!\n",result);
269         break;
270     case 16:
271         flagsave.clear();
272         printf("輸入檔案名: ");
273         scanf("%s",ch);
274         result=SaveGraph(G,ch);
275         if (result==OK) printf("OK\n");
276         else printf("ERROR! \n");
277         break;
```

华中科技大学课程实验报告

```
278         case 17:
279             printf("輸入檔案名: ");
280             scanf("%s",ch);
281             result=LoadGraph(G,ch);
282             if (result==OK) printf("OK\n");
283             else printf("ERROR! \n");
284             break;
285         case 18:
286             break;
287         case 0:
288             t=1;
289             break;
290             Lists.elem[result-1].G=G;
291             }//end of switch
292             system("PAUSE");
293             }
294             }
295             }
296             break;
297         case 4:
298             break;
299         case 0:
300             t=1;
301             break;
302     }
303     system("PAUSE");
304 }
305 break;
306
307 case 2:
308     op=1;
309     G=Gtemp;
310     while(op!=0 and op!=18){
311         int t=0;
312         system("cls"); printf("\n\n");
313         printf("    單圖管理!    \n");
314         printf("-----\n");
315         printf("        1. CreateCraph    7. InsertVex\n");
316         printf("        2. DestroyGraph 8. DeleteVex \n");
317         printf("        3. LocateVex    9. InsertArc \n");
318         printf("        4. PutVex    10. DeleteArc\n");
```



```
319         printf("      5. FirstAdjVex  11. DFSTraverse\n");
320         printf("      6. NextAdjVex   12. BFSTraverse\n");
321         printf("      13. VerticesSetLessThanK 14.
           ShortestPathLength\n");
322         printf("      15. ConnectedComponentsNums 16.
           SaveGraph\n");
323         printf("      17. LoadGraph  18. 返回多线性表操作\n");
324         printf("      0. Exit\n");
325         printf("-----\n");
326         printf("  请选择你的操作[0~18]:");
327         scanf("%d",&op);
328         switch(op){
329             case 1:
330                 if(G.vexnum!=0){
331                     printf("图已存在!");
332                     break;
333                 }
334                 printf("请输入定义序列:\n");
335                 i_1=0;
336                 do {
337                     scanf("%d%s",&V[i_1].key,V[i_1].others);
338                 } while(V[i_1++].key!=-1);
339                 i_1=0;
340                 do {
341                     scanf("%d%d",&VR[i_1][0],&VR[i_1][1]);
342                 } while(VR[i_1++][0]!=-1);
343                 result=CreateCraph(G,V,VR);
344                 if (result==OK) printf("OK\n");
345                 if (result==ERROR) printf("输入错误! \n");
346                 break;
347             case 2:
348                 result=DestroyGraph(G);
349                 if (result==OK) printf("OK\n");
350                 else printf("图不存在! \n");
351                 break;
352             case 3:
353                 printf("输入查找的關鍵字: ");
354                 scanf("%d",&tp);
355                 result=LocateVex(G,tp);
356                 if (result!=-1) printf("%d
           %s\n",G.vertices[result].data.key,G.vertices[result].data.others);
```

```
357         else printf("ERROR! \n");
358         break;
359     case 4:
360         printf("輸入關鍵字與修改為的值: ");
361         scanf("%d%d%s",&tp,&c.key,c.others);
362         result=PutVex(G,tp,c);
363         if (result==OK) printf("OK! \n");
364         else printf("ERROR! \n");
365         break;
366     case 5:
367         printf("輸入要查找的關鍵字: ");
368         scanf("%d",&tp);
369         result= FirstAdjVex(G,tp);
370         if (result== -1) printf("ERROR! \n");
371         else printf("%d
            %s\n",G.vertices[result].data.key,G.vertices[result].data.others);
372         break;
373     case 6:
374         printf("輸入v和w: ");
375         scanf("%d%d",&tp,&e);
376         result= NextAdjVex(G,tp,e);
377         if (result== -1) printf("ERROR! \n");
378         else printf("%d
            %s\n",G.vertices[result].data.key,G.vertices[result].data.others);
379         break;
380     case 7:
381         printf("輸入插入的點: ");
382         scanf("%d%s",&c.key,c.others);
383         result=InsertVex(G,c);
384         if(result == OK) printf("OK!\n");
385         else printf("ERROR! \n");
386         break;
387     case 8:
388         printf("輸入你要刪除的關鍵字: ");
389         scanf("%d",&e);
390         result=DeleteVex(G,e);
391         if (result==OK) printf("OK! \n");
392         else printf("ERROR!\n");
393         break;
394     case 9:
395         printf("輸入插入的弧<v,w>");
```

```
396         scanf("%d%d",&e,&tp);
397         result=InsertArc(G,e,tp);
398         if (result==OK) printf("OK! \n");
399         else printf("插入失败! \n");
400         break;
401     case 10:
402         printf("輸入插入的弧<v,w>");
403         scanf("%d%d",&e,&tp);
404         result=DeleteArc(G,e,tp);
405         if (result==OK) printf("OK! \n");
406         else printf("插入失败! \n");
407         break;
408     case 11:
409         flagdfs.clear();
410         DFSTraverse(G,visit);
411         break;
412     case 12:
413         flagbfs.clear();
414         while (!q.empty()) {
415             q.pop();
416         }
417         BFSTraverse(G,visit);
418         break;
419     case 13:
420         flagbfs.clear();
421         while (!q.empty()) {
422             q.pop();
423         }
424         printf("輸入v與k: ");
425         scanf("%d%d",&tp,&e);
426         result=VerticesSetLessThanK(G,tp,e);
427         if (result==ERROR) printf("頂點不存在! \n");
428         break;
429     case 14:
430
431         flagbfs.clear();
432         while (!q.empty()) {
433             q.pop();
434         }
435         printf("輸入v與w: ");
436         scanf("%d%d",&tp,&e);
```

```
437         result=ShortestPathLength(G,tp,e);
438         if (result==-1) printf("不存在! \n");
439         else printf("距離為%d! \n",result);
440         break;
441         case 15:
442             flagbfs.clear();
443             while (!q.empty()) {
444                 q.pop();
445             }
446             result=ConnectedComponentsNums(G);
447             if (result==-1) printf("圖不存在! \n");
448             else printf("連通分量有%d個! \n",result);
449             break;
450             case 16:
451                 flagsave.clear();
452                 printf("輸入檔案名: ");
453                 scanf("%s",ch);
454                 result=SaveGraph(G,ch);
455                 if (result==OK) printf("OK\n");
456                 else printf("ERROR! \n");
457                 break;
458                 case 17:
459                     printf("輸入檔案名: ");
460                     scanf("%s",ch);
461                     result=LoadGraph(G,ch);
462                     if (result==OK) printf("OK\n");
463                     else printf("ERROR! \n");
464                     break;
465                     case 18:
466                         Gtemp = G;
467                         break;
468                         case 0:
469                             t=1;
470                             break;
471             }//end of switch
472             system("PAUSE");
473         }
474
475         break;//case2 break
476         case 0:
477             break;
```

华中科技大学课程实验报告

```
478
479         }
480         if(t==1) break;
481
482     }//end of while
483     printf("歡迎下次再使用本系統! \n");
484     return 0;
485 }//end of main()
486 /*-----page 23 on textbook -----*/
487 status CreateGraph(ALGraph &G,VertexType V[],KeyType VR[][2])
488 /*根據V和VR構造圖T並返回OK, 如果V和VR不正確, 返回ERROR
489 如果有相同的關鍵字, 返回ERROR。此題允許通過增加其它函數輔助實現本關任務*/
490 {
491     // 請在這裡補充代碼, 完成本關任務
492     /****** Begin *****/
493     G.vexnum = 0;
494     while (G.vexnum <= MAX_VERTEX_NUM && V[G.vexnum].key != -1) { // 假設以-1結束
495         G.vexnum++;
496     }
497     if (G.vexnum > MAX_VERTEX_NUM or G.vexnum==0) return ERROR;
498
499     // 檢查頂點關鍵字唯一性
500     for (int i = 0; i < G.vexnum; ++i) {
501         for (int j = i + 1; j < G.vexnum; ++j) {
502             if (V[i].key == V[j].key)
503                 return ERROR;
504         }
505     }
506
507     // 初始化圖的頂點
508     for (int i = 0; i < G.vexnum; ++i) {
509         G.vertices[i].data = V[i];
510         G.vertices[i].firstarc = NULL;
511     }
512
513     // 計算邊數
514     G.arcnum = 0;
515     while (VR[G.arcnum][0] != -1 && VR[G.arcnum][1] != -1) { // 假設以-1結束
516         G.arcnum++;
517     }
518 }
```

```
519 // 處理每條邊
520 for (int k = 0; k < G.arcnum; ++k) {
521     KeyType u = VR[k][0];
522     KeyType v = VR[k][1];
523
524     int i = LocateVex(G, u);
525     int j = LocateVex(G, v);
526     if (i == -1 || j == -1)
527         return ERROR;
528
529     // 將j添加到i的鄰接表中
530     ArcNode *arc_i = (ArcNode *)malloc(sizeof(ArcNode));
531     if (!arc_i) return OVERFLOW;
532     arc_i->adjvex = j;
533     arc_i->nextarc = G.vertices[i].firstarc;
534     G.vertices[i].firstarc = arc_i;
535
536     // 將i添加到j的鄰接表中
537     ArcNode *arc_j = (ArcNode *)malloc(sizeof(ArcNode));
538     if (!arc_j) {
539         free(arc_i);
540         return OVERFLOW;
541     }
542     arc_j->adjvex = i;
543     arc_j->nextarc = G.vertices[j].firstarc;
544     G.vertices[j].firstarc = arc_j;
545 }
546
547 G.kind = UDG;
548 return OK;
549 /***** End *****/
550 }
551
552 status DestroyGraph(ALGraph &G)
553 /*銷毀無向圖G,刪除G的全部頂點和邊*/
554 {
555     // 請在這裡補充代碼,完成本關任務
556     /***** Begin *****/
557     if (G.vexnum==0) return ERROR;
558     for (int i = 0; i < G.vexnum; ++i) {
559         ArcNode *p = G.vertices[i].firstarc;
```

```
560         while (p != NULL) {
561             ArcNode *temp = p;
562             p = p->nextarc;
563             free(temp);
564         }
565         G.vertices[i].firstarc = NULL;
566     }
567     G.vexnum = 0;
568     G.arcnum = 0;
569
570
571     return OK;
572
573     ***** End *****
574 }
575
576 int LocateVex(ALGraph G,KeyType u)
577 //根據u在圖G中查找頂點，查找成功返回位序，否則返回-1;
578 {
579     // 請在這裡補充代碼，完成本關任務
580     ***** Begin *****
581     for (int i = 0; i < G.vexnum; ++i) {
582         if(G.vertices[i].data.key == u) return i;
583     }
584     return -1;
585
586     ***** End *****
587 }
588
589 int checkput(ALGraph G,int x,VertexType v){
590     for (int i = 0; i < G.vexnum; ++i) {
591         if(i == x) continue ;
592         if(v.key == G.vertices[i].data.key) return 0;
593     }
594     return 1;
595 }
596
597 status PutVex(ALGraph &G,KeyType u,VertexType value)
598 //根據u在圖G中查找頂點，查找成功將該頂點值修改成value，返回OK;
599 //如果查找失敗或關鍵字不唯一，返回ERROR
600 {
601     // 請在這裡補充代碼，完成本關任務
```

```
601      ***** Begin *****
602      for (int i = 0; i < G.vexnum; ++i) {
603          if(G.vertices[i].data.key == u){
604              if(checkput(G,i,value)== 1){
605                  G.vertices[i].data=value;
606                  return OK;
607              }
608              else return ERROR;
609          }
610      }
611      return ERROR;
612
613      ***** End *****
614  }
615
616  int FirstAdjVex(ALGraph G,KeyType u)
617  //根據u在圖G中查找頂點，查找成功返回頂點u的第一鄰接頂點位序，否則返回-1;
618  {
619      // 請在這裡補充代碼，完成本關任務
620      ***** Begin *****
621      int t = LocateVex(G,u);
622      if(G.vertices[t].firstarc!=NULL) return G.vertices[t].firstarc->adjvex;
623      return -1;
624      ***** End *****
625  }
626
627  int NextAdjVex(ALGraph G,KeyType v,KeyType w)
628  //v對應G的一個頂點,w對應v的鄰接頂點；操作結果是返回v的（相對於w）下一個鄰接頂點的位序；如果w是最後一個鄰接頂點，
629  {
630      // 請在這裡補充代碼，完成本關任務
631      ***** Begin *****
632      int t = LocateVex(G,v);
633      int wt = LocateVex(G,w);
634      if (t==-1 or wt == -1) return -1;
635      ArcNode *p = G.vertices[t].firstarc;
636      while(p!=NULL){
637          if(p->adjvex==wt){
638              if(p->nextarc!=NULL) return p->nextarc->adjvex;
639              else return -1;
640          }
641          p=p->nextarc;
```



```
642     }
643     return -1;
644
645     /****** End *****/
646 }
647
648 status InsertVex(ALGraph &G,VertexType v)
649 //在圖G中插入頂點v, 成功返回OK,否則返回ERROR
650 {
651     // 請在這裡補充代碼, 完成本關任務
652     /****** Begin *****/
653     if(G.vexnum == MAX_VERTEX_NUM) return ERROR;
654     for(int i=0;i<G.vexnum;i++){
655         if(G.vertices[i].data.key==v.key) return ERROR;
656     }
657     G.vertices[G.vexnum].data = v;
658     G.vertices[G.vexnum].firstarc=NULL;
659     G.vexnum++;
660     return OK;
661     /****** End *****/
662 }
663
664 status DeleteVex(ALGraph &G,KeyType v)
665 //在圖G中刪除關鍵字v對應的頂點以及相關的弧, 成功返回OK,否則返回ERROR
666 {
667     // 請在這裡補充代碼, 完成本關任務
668     /****** Begin *****/
669     if(G.vexnum == 1) return ERROR;
670     int pos_v = LocateVex(G, v);
671     if (pos_v == -1) return ERROR;
672
673     ArcNode *current = G.vertices[pos_v].firstarc;
674     while (current != NULL) {
675         int w_pos = current->adjvex;
676
677         ArcNode *w_prev = NULL;
678         ArcNode *w_p = G.vertices[w_pos].firstarc;
679         while (w_p != NULL) {
680             if (w_p->adjvex == pos_v) {
681                 if (w_prev == NULL) {
682                     G.vertices[w_pos].firstarc = w_p->nextarc;
```

```
683         } else {
684             w_prev->nextarc = w_p->nextarc;
685         }
686         ArcNode *temp = w_p;
687         w_p = w_p->nextarc;
688         free(temp);
689         G.arcnum--;
690         break;
691     } else {
692         w_prev = w_p;
693         w_p = w_p->nextarc;
694     }
695 }
696
697 ArcNode *temp = current;
698 current = current->nextarc;
699 free(temp);
700 }
701 G.vertices[pos_v].firstarc = NULL;
702
703 for (int i = pos_v; i < G.vexnum - 1; i++) {
704     G.vertices[i] = G.vertices[i + 1];
705 }
706 G.vexnum--;
707 for (int i = 0; i < G.vexnum; i++) {
708     ArcNode *p = G.vertices[i].firstarc;
709     while (p != NULL) {
710         if (p->adjvex > pos_v) {
711             p->adjvex--;
712         }
713         p = p->nextarc;
714     }
715 }
716
717 return OK;
718 /***** End *****/
719 }
720
721 status InsertArc(ALGraph &G,KeyType v,KeyType w)
722 //在圖G中增加弧<v,w>, 成功返回OK, 否則返回ERROR
723 {
```

```
724 // 請在這裡補充代碼，完成本關任務
725 /***** Begin *****/
726 int vt = LocateVex(G,v),wt = LocateVex(G,w);
727 if(vt == -1 or wt == -1) return ERROR;
728 ArcNode *p = G.vertices[wt].firstarc;
729 while(p!=NULL){
730     if(p->adjvex == vt) return ERROR;
731     p=p->nextarc;
732 }
733 ArcNode *temp1 = (ArcNode *)malloc(sizeof(ArcNode));
734 temp1->nextarc = G.vertices[vt].firstarc;
735 temp1->adjvex = wt;
736 G.vertices[vt].firstarc = temp1;
737
738 ArcNode *temp2 = (ArcNode *)malloc(sizeof(ArcNode));
739 temp2->nextarc = G.vertices[wt].firstarc;
740 temp2->adjvex = vt;
741 G.vertices[wt].firstarc = temp2;
742
743 G.arcnum++;
744 return OK;
745 /***** End *****/
746 }
747
748 status DeleteArc(ALGraph &G,KeyType v,KeyType w)
749 //在圖G中刪除弧<v,w>, 成功返回OK, 否則返回ERROR
750 {
751     // 請在這裡補充代碼，完成本關任務
752     /***** Begin *****/
753     int v_pos = LocateVex(G, v);
754     int w_pos = LocateVex(G, w);
755     if (v_pos == -1 || w_pos == -1) return ERROR;
756
757     ArcNode *prev = NULL;
758     ArcNode *curr = G.vertices[v_pos].firstarc;
759     bool found = false;
760     while (curr != NULL) {
761         if (curr->adjvex == w_pos) {
762             if (prev == NULL) {
763                 G.vertices[v_pos].firstarc = curr->nextarc;
764             } else {
```

```
765         prev->nextarc = curr->nextarc;
766     }
767     free(curr);
768     found = true;
769     break;
770 }
771 prev = curr;
772 curr = curr->nextarc;
773 }
774 if (!found) return ERROR;
775
776 prev = NULL;
777 curr = G.vertices[w_pos].firstarc;
778 while (curr != NULL) {
779     if (curr->adjvex == v_pos) {
780         if (prev == NULL) {
781             G.vertices[w_pos].firstarc = curr->nextarc;
782         } else {
783             prev->nextarc = curr->nextarc;
784         }
785         free(curr);
786         break;
787     }
788     prev = curr;
789     curr = curr->nextarc;
790 }
791
792 G.arcnum--;
793 return OK;
794
795 /***** End *****/
796 }
797
798 void dfs(ALGraph G,int t,void (*visit)(VertexType)){
799     visit(G.vertices[t].data);
800     flagdfs[t]=1;
801     ArcNode *p = G.vertices[t].firstarc;
802     while(p!=NULL){
803         if(flagdfs[p->adjvex]==0) dfs(G,p->adjvex,visit);
804         p=p->nextarc;
805     }
```

华中科技大学课程实验报告

```
806 }
807
808 status DFSTraverse(ALGraph &G,void (*visit)(VertexType))
809 //對圖G進行深度優先搜索遍歷，依次對圖中的每一個頂點使用函數visit訪問一次，且僅訪問一次
810 {
811     // 請在這裡補充代碼，完成本關任務
812     /***** Begin *****/
813     for(int i=0;i<G.vexnum;i++){
814         if(flagdfs[i]==0){
815             dfs(G,i,visit);
816             printf("\n");
817         }
818     }
819
820     /***** End *****/
821 }
822
823 status BFSTraverse(ALGraph &G,void (*visit)(VertexType))
824 //對圖G進行廣度優先搜索遍歷，依次對圖中的每一個頂點使用函數visit訪問一次，且僅訪問一次
825 {
826     // 請在這裡補充代碼，完成本關任務
827     /***** Begin *****/
828     for(int i=0;i<G.vexnum;i++){
829         if(flagbfs[i]==0){
830             q.push(i);
831             flagbfs[i]=1;
832         }
833         else continue;
834         while(!q.empty()){
835             int u=q.front();
836             q.pop();
837             visit(G.vertices[u].data);
838             ArcNode *p = G.vertices[u].firstarc;
839             while(p!=NULL){
840                 if(flagbfs[p->adjvex]==0){
841                     q.push(p->adjvex);
842                     flagbfs[p->adjvex]=1;
843                 }
844                 p=p->nextarc;
845             }
846         }
```

```
847         printf("\n");
848     }
849
850     /****** End *****/
851 }
852 int VerticesSetLessThanK(ALGraph &G,int v,int k){
853     int u0= LocateVex(G,v),s[100]={-1};
854     if(u0==-1) return ERROR;
855     q.push(u0);
856     s[u0]=0;
857     flagbfs[u0]=1;
858     printf("%d %s\n",G.vertices[u0].data.key,G.vertices[u0].data.others);
859     while(!q.empty()){
860         int u=q.front();
861         q.pop();
862         ArcNode *p = G.vertices[u].firstarc;
863         while(p!=NULL){
864             if(flagbfs[p->adjvex]==0){
865                 s[p->adjvex]=s[u]+1;
866                 if(s[p->adjvex]<k){
867                     flagbfs[p->adjvex]=1;
868                     q.push(p->adjvex);
869                     printf("%d
                        %s\n",G.vertices[p->adjvex].data.key,G.vertices[p->adjvex].data.others);
870                 }
871             }
872             p=p->nextarc;
873         }
874     }
875     return OK;
876 }
877
878 int ShortestPathLength(ALGraph &G,int v,int k){
879     int u0= LocateVex(G,v),s[100]={-1},v0=LocateVex(G,k);
880     if(u0==-1 or v0==-1) return -1;
881     q.push(u0);
882     s[u0]=0;
883     flagbfs[u0]=1;
884     while(!q.empty()){
885         int u=q.front();
886         q.pop();
```

```
887         ArcNode *p = G.vertices[u].firstarc;
888         while(p!=NULL){
889             if(flagbfs[p->adjvex]==0){
890                 s[p->adjvex]=s[u]+1;
891                 if(p->adjvex==v0){
892                     return s[p->adjvex];
893                 }
894                 flagbfs[p->adjvex]=1;
895                 q.push(p->adjvex);
896             }
897             p=p->nextarc;
898         }
899     }
900     return ERROR;
901 }
902
903 int ConnectedComponentsNums(ALGraph &G){
904     if(G.vexnum==0) return -1;
905     int s=0;
906     for(int i=0;i<G.vexnum;i++){
907         if(flagbfs[i]==0){
908             q.push(i);
909             flagbfs[i]=1;
910         }
911         else continue;
912         while(!q.empty()){
913             int u=q.front();
914             q.pop();
915             ArcNode *p = G.vertices[u].firstarc;
916             while(p!=NULL){
917                 if(flagbfs[p->adjvex]==0){
918                     q.push(p->adjvex);
919                     flagbfs[p->adjvex]=1;
920                 }
921                 p=p->nextarc;
922             }
923         }
924         s++;
925     }
926     return s;
927 }
```

```
928
929 status SaveGraph(ALGraph G, char FileName[])
930 //將圖的資料寫入到檔FileName中
931 {
932     // 請在這裡補充代碼，完成本關任務
933     /***** Begin 1 *****/
934     FILE *fp = fopen(FileName, "w");
935     for(int i=0;i<G.vexnum;i++){
936         fprintf(fp,"%d %s ", G.vertices[i].data.key , G.vertices[i].data.others);
937     }
938     fprintf(fp,"%d %s ", -1 , "nil");
939     for(int i=0;i<G.vexnum;i++){
940         flagsave[i]=1;
941         ArcNode *p = G.vertices[i].firstarc;
942         while(p!=NULL){
943             if(flagsave[p->adjvex] == 0){
944                 fprintf(fp,"%d %d ", G.vertices[i].data.key ,
945                     G.vertices[p->adjvex].data.key);
946             }
947             p=p->nextarc;
948         }
949         fprintf(fp,"%d %d ", -1 , -1);
950         fclose(fp);
951         return OK;
952     }
953     /***** End 1 *****/
954 }
955
956 status LoadGraph(ALGraph &G, char FileName[])
957 //讀入檔FileName的圖資料，創建圖的鄰接表
958 {
959     // 請在這裡補充代碼，完成本關任務
960     /***** Begin 2 *****/
961     FILE *fp = fopen(FileName, "r");
962     VertexType V[21];
963     KeyType VR[100][2];
964     int i=0;
965     do {
966         fscanf(fp,"%d%s",&V[i].key,V[i].others);
967     } while(V[i++].key!=-1);
```



```
968     i=0;
969     do {
970         fscanf(fp,"%d%d",&VR[i][0],&VR[i][1]);
971         if(VR[i][0]>VR[i][1]) std::swap(VR[i][0],VR[i][1]);
972     } while(VR[i++][0]!=-1);
973     i--;
974     for(int j=0;j<i-1;j++){
975         for(int r=0;r<i-1-j;r++)
976             if(VR[r][0] > VR[r+1][0] || (VR[r][0] == VR[r+1][0] && VR[r][1] >
977                 VR[r+1][1])){
978                 std::swap(VR[r],VR[r+1]);
979             }
980     }
981     if(CreateCraph(G,V,VR)==OK) {
982         fclose(fp);
983         return OK;
984     }
985     else{
986         fclose(fp);
987         return ERROR;
988     }
989     /***** End 2 *****/
990
991 status AddList(LISTS &Lists,char ListName[])
992 // 只需要在Lists中增加一个名称为ListName的空线性表，线性表资料又後臺測試程式插入。
993 {
994     // 請在這裡補充代碼，完成本關任務
995     /***** Begin *****/
996     Lists.elem[Lists.length].G = ALGraph(); // C++ 方式初始化
997     ALGraph &graph = Lists.elem[Lists.length].G;
998     graph.vexnum = 0;
999     graph.arcnum = 0;
1000    graph.kind = UDG;
1001    for (int i = 0; i < MAX_VERTEX_NUM; i++) {
1002        graph.vertices[i].firstarc = NULL;
1003    }
1004    strcpy(Lists.elem[Lists.length].name,ListName);
1005    Lists.length++;
1006    return OK;
1007    /***** End *****/
```

```
1008 }
1009 status RemoveList(LISTS &Lists, char ListName[])
1010 // Lists中删除一个名称为ListName的线性表
1011 {
1012     // 請在這裡補充代碼，完成本關任務
1013     /***** Begin *****/
1014     for(int i=0; i<Lists.length; i++){
1015         if(strcmp(ListName, Lists.elem[i].name)==0){
1016             for(int j=i; j<Lists.length-1; j++){
1017                 Lists.elem[j]=Lists.elem[j+1];
1018             }
1019             Lists.length--;
1020             return OK;
1021         }
1022     }
1023     return ERROR;
1024     /***** End *****/
1025 }
1026 int LocateList(LISTS Lists, char ListName[])
1027 // 在Lists中查找一个名称为ListName的线性表，成功返回逻辑序号，否则返回0
1028 {
1029     // 請在這裡補充代碼，完成本關任務
1030     /***** Begin *****/
1031     for(int i=0; i<Lists.length; i++){
1032         if(strcmp(Lists.elem[i].name, ListName)==0) return i+1;
1033     }
1034     return ERROR;
1035     /***** End *****/
1036 }
```